



**Universidad**  
Zaragoza



**Facultad de Ciencias**  
**Universidad** Zaragoza

UNIVERSIDAD DE ZARAGOZA

MÁSTER EN MODELIZACIÓN E INVESTIGACIÓN MATEMÁTICA,  
ESTADÍSTICA Y COMPUTACIÓN

---

TRABAJO DE FIN DE MÁSTER

# Predicción de Impago de Crédito mediante Técnicas de Machine Learning

*Credit Default Prediction Using Machine Learning Techniques*

---

**Autor:** Frank Salomón Sulca Palomino

**Director:** Dr. Ricardo López Ruiz

**Codirector:** Dr. Jorge Delgado Gracia

Año académico 2025–2026

# Índice general

|                                                                         |           |
|-------------------------------------------------------------------------|-----------|
| <b>1. Introducción</b>                                                  | <b>3</b>  |
| 1.1. Contexto del riesgo de crédito                                     | 3         |
| 1.2. Importancia de la predicción de impago y del scoring crediticio    | 3         |
| 1.3. Descripción general del problema a resolver                        | 4         |
| 1.4. Objetivos del trabajo                                              | 5         |
| 1.4.1. Objetivo general                                                 | 5         |
| 1.4.2. Objetivos específicos                                            | 5         |
| 1.5. Estructura del documento                                           | 7         |
| <b>2. Machine Learning</b>                                              | <b>8</b>  |
| 2.1. Aprendizaje supervisado                                            | 8         |
| 2.1.1. Regresión                                                        | 8         |
| 2.1.2. Clasificación                                                    | 9         |
| 2.2. Métricas de evaluación                                             | 12        |
| 2.2.1. Accuracy, Precision, Recall y F1                                 | 12        |
| 2.2.2. Curva ROC y ROC-AUC                                              | 13        |
| 2.2.3. Curva Precision-Recall y PR-AUC                                  | 13        |
| 2.3. Desbalance de clases                                               | 14        |
| 2.4. Validación cruzada y Stratified K-Fold                             | 14        |
| <b>3. Árboles de decisión y métodos de ensamblado</b>                   | <b>16</b> |
| 3.1. Árboles de decisión                                                | 16        |
| 3.2. Bagging                                                            | 17        |
| 3.3. Random Forest                                                      | 18        |
| <b>4. Boosting y XGBoost</b>                                            | <b>20</b> |
| 4.1. Idea general del boosting                                          | 20        |
| 4.2. Gradient Boosting                                                  | 20        |
| 4.3. XGBoost                                                            | 22        |
| 4.4. Parámetros principales y control del sobreajuste                   | 23        |
| <b>5. Aplicación práctica: Predicción y scoring de riesgo de impago</b> | <b>25</b> |
| 5.1. Introducción al dataset Home Credit Default Risk                   | 25        |
| 5.2. Preparación y exploración de datos                                 | 25        |
| 5.3. Diseño experimental y estrategia de evaluación                     | 27        |
| 5.4. Interpretación del desbalanceo                                     | 29        |
| 5.5. Enfoque aplicado del scoring y valor diferencial del análisis      | 30        |
| 5.6. Comparación inicial de modelos mediante PR-AUC y ROC-AUC           | 32        |

|                                                                                 |           |
|---------------------------------------------------------------------------------|-----------|
| 5.7. Evaluación de estrategias frente al desbalanceo en XGBoost . . . . .       | 33        |
| 5.8. Selección, ajuste y análisis de umbrales del modelo seleccionado . . . . . | 34        |
| 5.9. Evaluación final del modelo sobre el conjunto de test . . . . .            | 36        |
| 5.10. Evaluación del scoring: deciles, lift y captura acumulada . . . . .       | 37        |
| <b>6. Conclusiones</b>                                                          | <b>40</b> |
| 6.1. Principales resultados . . . . .                                           | 40        |
| 6.2. Utilidad práctica del modelo . . . . .                                     | 41        |
| 6.3. Limitaciones del estudio . . . . .                                         | 41        |
| 6.4. Líneas futuras de investigación . . . . .                                  | 42        |
| <b>Anexos</b>                                                                   | <b>43</b> |
| <b>A. Resultados completos de estrategias frente al desbalanceo</b>             | <b>43</b> |
| <b>B. Código relevante del capítulo 5</b>                                       | <b>44</b> |
| B.1. Partición estratificada y creación de variables . . . . .                  | 44        |
| B.2. Preprocesamiento dentro de cada fold . . . . .                             | 45        |
| B.3. Validación cruzada y métricas de evaluación . . . . .                      | 46        |
| B.4. Ajuste práctico del modelo XGBoost . . . . .                               | 47        |
| B.5. Umbrales, deciles, lift y captura acumulada . . . . .                      | 49        |
| <b>Bibliografía</b>                                                             | <b>51</b> |

# 1. Introducción

## 1.1. Contexto del riesgo de crédito

El riesgo de crédito es uno de los aspectos centrales en la gestión financiera de bancos, entidades prestamistas y organizaciones que conceden financiación a consumidores. En términos generales, este riesgo se refiere a la posibilidad de que un cliente no cumpla con el pago de sus obligaciones financieras. Por tanto, toda decisión de concesión de crédito implica un grado de incertidumbre, ya que no siempre es posible saber con certeza si el solicitante cumplirá sus pagos en el futuro [1].

Esta incertidumbre tiene implicaciones directas sobre la rentabilidad de las entidades, la gestión de provisiones, la planificación del capital y los sistemas internos de control. Por ello, disponer de herramientas que ayuden a estimar el riesgo de impago puede aportar valor en los procesos de análisis crediticio, especialmente cuando el volumen de solicitudes es elevado y no todos los casos pueden revisarse con la misma intensidad [1, 2].

En los últimos años, el aumento del volumen de datos disponibles sobre clientes, operaciones financieras e historiales crediticios ha favorecido la aplicación de técnicas de aprendizaje automático en este ámbito. Estos modelos permiten identificar patrones en los datos y estimar la probabilidad de que un cliente incurra en impago. Sin embargo, en riesgo de crédito no basta con construir un modelo que produzca una predicción; también es necesario analizar cómo podría utilizarse esa predicción dentro de un proceso de negocio [2].

Una característica importante de este tipo de problemas es el desbalanceo de clases. Normalmente, la mayoría de clientes cumple con sus pagos, mientras que solo una proporción reducida incurre en impago. Esto provoca que algunas métricas generales puedan ofrecer una visión incompleta del rendimiento del modelo. Por ejemplo, una accuracy elevada puede obtenerse incluso si el modelo identifica mal a la clase minoritaria, que es precisamente la más relevante desde el punto de vista del riesgo [3, 4].

Por esta razón, el análisis del riesgo de crédito requiere evaluar no solo si el modelo acierta una etiqueta final, sino también si es capaz de ordenar a los clientes según su nivel relativo de riesgo. En este contexto, una puntuación de riesgo puede ser más útil que una decisión automática, ya que permite segmentar clientes, priorizar revisiones y concentrar los recursos de análisis en aquellos grupos donde la probabilidad de impago es mayor [1, 2].

## 1.2. Importancia de la predicción de impago y del scoring crediticio

La predicción de impago es relevante porque permite anticipar posibles pérdidas, mejorar la asignación de recursos y apoyar los procesos de concesión, seguimiento y control del crédito. No obstante, en una aplicación real, un modelo de aprendizaje automático no

debería interpretarse necesariamente como una herramienta que decide por sí sola si un crédito debe aprobarse o rechazarse.

En este trabajo, el modelo se interpreta principalmente como una herramienta de *scoring*. Es decir, se busca asignar a cada cliente una puntuación o probabilidad estimada de impago, de forma que las solicitudes puedan ordenarse de mayor a menor riesgo. Este enfoque resulta especialmente útil cuando el objetivo no es automatizar la decisión final, sino apoyar al analista en la identificación de los casos que requieren mayor atención [1, 2].

Esta distinción es importante en problemas desbalanceados. Si los impagos representan una proporción reducida del total, es esperable que la *precision* del modelo no sea muy elevada, ya que muchos clientes señalados como riesgo finalmente no incurrirán en impago. Por ello, una baja *precision* limita el uso del modelo como mecanismo automático de rechazo, pero no implica necesariamente que el score carezca de valor. El modelo puede seguir siendo útil si permite identificar segmentos de clientes con una tasa de impago claramente superior a la tasa media de la población [3, 4].

Desde esta perspectiva, el valor de un modelo de scoring no debe evaluarse únicamente mediante métricas globales como ROC-AUC. Aunque ROC-AUC permite medir la capacidad general del modelo para ordenar clientes con impago por encima de clientes sin impago, en un contexto de fuerte desbalanceo también es necesario considerar métricas más orientadas a la clase positiva y a la utilidad práctica del ranking. Por ello, este trabajo incorpora PR-AUC, *precision*, *recall*, análisis de umbrales, deciles de riesgo, lift y captura acumulada de impagos [5, 6, 4].

Estas herramientas permiten responder preguntas más cercanas al negocio, como qué proporción de clientes marcados como riesgo realmente impaga, cuántos impagos se capturan revisando solo los segmentos superiores del ranking, o en qué medida los primeros deciles concentran más riesgo que la población promedio. De esta manera, la evaluación del modelo no se limita a comprobar si predice mejor o peor, sino que analiza si su score puede utilizarse de forma práctica y responsable como apoyo a la decisión.

### 1.3. Descripción general del problema a resolver

En este trabajo se utiliza el conjunto de datos *Home Credit Default Risk*, que contiene información socioeconómica, financiera y crediticia de clientes que solicitan préstamos. La variable objetivo indica si el cliente incurrió o no en impago, por lo que el problema puede formularse inicialmente como una tarea de clasificación binaria supervisada [7].

Sin embargo, debido al fuerte desbalanceo entre clientes cumplidores e incumplidores, el problema no se aborda únicamente como una clasificación 0/1. El objetivo aplicado es analizar si los modelos pueden generar una puntuación de riesgo útil para ordenar a los clientes y priorizar aquellos casos donde la probabilidad de impago es mayor.

El conjunto de datos *Home Credit Default Risk* ha sido ampliamente utilizado en competiciones y trabajos previos, donde el interés principal suele centrarse en mejorar el rendimiento predictivo mediante métricas globales como ROC-AUC [7]. En este trabajo, sin embargo, el objetivo no es competir con las soluciones más optimizadas sobre esta base, sino evaluar la utilidad práctica del modelo como herramienta de scoring en un

contexto de fuerte desbalanceo de clases.

Para ello, se comparan distintos modelos y estrategias de tratamiento del desbalanceo, incluyendo la asignación de mayor peso a la clase minoritaria, técnicas de remuestreo y ajuste de hiperparámetros. No obstante, el interés principal no está únicamente en identificar el modelo con mejor métrica global, sino en valorar qué estrategias generan un ranking más útil, estable e interpretable desde el punto de vista del negocio [3, 2].

De esta forma, el trabajo busca responder una pregunta práctica: si una entidad no puede revisar todas las solicitudes con el mismo nivel de detalle, ¿puede un modelo de aprendizaje automático ayudar a identificar qué clientes deberían revisarse primero? La respuesta a esta pregunta se estudia mediante PR-AUC, ROC-AUC, deciles de riesgo, lift, captura acumulada de impagos y análisis de umbrales.

## 1.4. Objetivos del trabajo

### 1.4.1. Objetivo general

El objetivo general de este trabajo es desarrollar y evaluar modelos de aprendizaje automático para estimar el riesgo de impago crediticio, analizando no solo su capacidad predictiva global, sino especialmente la utilidad práctica del *score* generado como herramienta de ordenación, segmentación y priorización de clientes en un contexto de fuerte desbalanceo de clases.

El trabajo no se plantea como una búsqueda del modelo con mayor exactitud global, ni como un sistema automático de aprobación o rechazo de créditos. Su finalidad es estudiar si las probabilidades estimadas por el modelo permiten ordenar a los clientes según su riesgo relativo y concentrar una mayor proporción de impagos en los segmentos superiores del ranking.

Desde esta perspectiva, el valor del modelo se interpreta en términos de apoyo a la decisión. En particular, se busca evaluar si el *score* puede ayudar a priorizar revisiones, asignar recursos de análisis a los clientes con mayor riesgo estimado y complementar el criterio del analista en procesos de evaluación crediticia. Al mismo tiempo, se reconocen sus limitaciones, especialmente las derivadas del desbalanceo de la variable objetivo, la presencia de falsos positivos y la baja *precision* asociada a ciertos umbrales de clasificación.

### 1.4.2. Objetivos específicos

Para alcanzar este objetivo general, se plantean los siguientes objetivos específicos:

En primer lugar, explorar el conjunto de datos *Home Credit Default Risk*, analizando la variable objetivo, la distribución de clientes con y sin impago, la presencia de valores faltantes y las principales variables disponibles. Esta etapa permite contextualizar el problema y justificar la necesidad de utilizar métricas adecuadas para datos desbalanceados, en lugar de basar la evaluación únicamente en la *accuracy*.

En segundo lugar, construir un flujo metodológico reproducible que incluya la preparación del conjunto de datos, la creación de variables explicativas, el tratamiento de valores ausentes y la codificación de variables categóricas. Asimismo, se define una separación

entre un conjunto de desarrollo y un conjunto de test final, de modo que la evaluación final se realice sobre datos no utilizados previamente en la selección del modelo.

En tercer lugar, comparar distintos modelos de clasificación supervisada, incluyendo regresión logística, Random Forest y XGBoost. Esta comparación permite establecer una referencia inicial de rendimiento y seleccionar modelos candidatos en función de su capacidad discriminante, su comportamiento frente a la clase minoritaria y su estabilidad en validación cruzada.

En cuarto lugar, evaluar distintas estrategias frente al desbalanceo de clases, como la ponderación de la clase positiva, el ajuste de `scale_pos_weight` y técnicas de remuestreo. El objetivo no es asumir que estas técnicas mejoran automáticamente el rendimiento, sino analizar empíricamente cómo afectan al equilibrio entre *precision*, *recall*, F1-score, PR-AUC y ROC-AUC.

En quinto lugar, utilizar PR-AUC como métrica principal de comparación, dado que la clase positiva corresponde a los clientes con impago y representa una proporción reducida del total. ROC-AUC se emplea como métrica complementaria para evaluar la capacidad general de discriminación, mientras que *precision*, *recall* y F1-score se interpretan principalmente en función del umbral de clasificación considerado.

En sexto lugar, realizar un ajuste ligero de hiperparámetros sobre el modelo candidato, con especial atención a las configuraciones asociadas a la ponderación de la clase positiva. Esta etapa busca seleccionar una configuración final que mantenga un rendimiento estable y conserve una capacidad adecuada para ordenar a los clientes según su riesgo estimado de impago.

En séptimo lugar, analizar el efecto de distintos umbrales de clasificación sobre las métricas del modelo. Este análisis permite observar cómo cambia el equilibrio entre *precision* y *recall* al modificar el punto de corte, mostrando que el uso operativo del modelo puede adaptarse a distintos criterios, como detectar más casos de impago o reducir el número de falsas alertas.

En octavo lugar, evaluar el modelo final sobre el conjunto de test reservado. Esta evaluación permite comprobar si el comportamiento observado durante la validación se mantiene sobre datos no utilizados en las etapas previas de selección, ajuste o análisis de umbrales.

En noveno lugar, estudiar la utilidad del modelo como herramienta de *scoring*. Para ello, los clientes del conjunto de test se ordenan según la probabilidad estimada de impago y se analizan los deciles de riesgo, el *lift* y la captura acumulada de impagos. Este análisis permite valorar si el modelo concentra efectivamente una mayor proporción de clientes incumplidores en los segmentos de mayor riesgo.

Finalmente, sintetizar los resultados obtenidos y discutir las principales limitaciones del enfoque propuesto. En particular, se analiza por qué una *precision* moderada o baja limita el uso del modelo como mecanismo automático de decisión, pero no elimina necesariamente su utilidad como herramienta de priorización y apoyo al análisis crediticio.

## 1.5. Estructura del documento

El trabajo se organiza en varios capítulos que permiten avanzar de manera progresiva desde los conceptos generales hasta la aplicación práctica. Tras esta introducción, el segundo capítulo presenta los fundamentos del aprendizaje automático necesarios para contextualizar el análisis. Se explican las nociones básicas del aprendizaje supervisado, las principales métricas de evaluación empleadas en problemas de clasificación y ranking, y la importancia de la validación cruzada, prestando especial atención a los problemas de clases desbalanceadas.

El tercer capítulo se centra en los árboles de decisión y en los modelos de ensamblado basados en árboles. En primer lugar, se describe la lógica de partición binaria recursiva, los criterios de impureza utilizados para seleccionar divisiones, como Gini y entropía, y algunos mecanismos de control del sobreajuste. Posteriormente, se presentan el bagging y Random Forest como métodos que combinan múltiples árboles para mejorar la estabilidad y la capacidad predictiva.

A continuación, el cuarto capítulo profundiza en los métodos de boosting, destacando el modelo XGBoost por su eficiencia y capacidad para manejar datos complejos. Este capítulo permite conectar la lógica de los árboles de decisión con los métodos de ensamblado secuencial utilizados posteriormente en la parte práctica del trabajo.

El quinto capítulo constituye la parte aplicada del trabajo. En él se analiza el conjunto de datos *Home Credit Default Risk*, se aplican los procesos de transformación y preprocesamiento necesarios, y se comparan distintos modelos de clasificación mediante un flujo experimental basado en validación cruzada. Además, se estudia el desbalanceo de clases, se comparan estrategias de tratamiento del desbalanceo, se realiza un ajuste ligero del modelo candidato y se analiza su utilidad como herramienta de scoring mediante PR-AUC, ROC-AUC, deciles de riesgo, lift, captura acumulada y análisis de umbrales. Finalmente, el modelo seleccionado se evalúa sobre un conjunto de test independiente.

El sexto capítulo recoge las conclusiones generales del trabajo, resume los hallazgos más relevantes y plantea posibles líneas futuras de investigación. Junto con los anexos y la bibliografía, este capítulo cierra el documento y ofrece una visión global de la contribución realizada.



## 2. Machine Learning

### 2.1. Aprendizaje supervisado

Dentro del campo del *Machine Learning*, una de las ramas más importantes es el **aprendizaje supervisado**. La idea básica es trabajar con datos donde, para cada observación, conocemos tanto las variables explicativas  $X$  como la respuesta  $Y$ . Es decir, el modelo aprende a partir de ejemplos del tipo:

$$(X_1, Y_1), (X_2, Y_2), \dots, (X_n, Y_n),$$

con el objetivo de aproximar la relación entre  $X$  e  $Y$  y poder predecir  $Y$  para nuevas observaciones [8, 9].

Desde un punto de vista más estadístico, lo que buscamos es modelar el **valor esperado condicional** de la respuesta dado el conjunto de predictores:

$$\mu = \mathbb{E}(Y \mid X).$$

Esta expresión representa el comportamiento medio de la variable respuesta condicionado a los predictores. En la práctica, dicha esperanza condicional se aproxima mediante modelos de aprendizaje automático, que aprenden una función a partir de los datos observados [9].

En función del tipo de variable respuesta  $Y$ , distinguimos principalmente dos grandes problemas:

- **Regresión:** cuando  $Y$  es una variable continua, por ejemplo, un importe, una puntuación o un tiempo.
- **Clasificación:** cuando  $Y$  es una variable discreta, normalmente categórica o binaria, por ejemplo, *paga / no paga* o *impago / no impago*.

En este trabajo, la variable respuesta es binaria, ya que indica si un cliente incurrió o no en impago. Por tanto, desde el punto de vista metodológico, el problema se enmarca dentro de la **clasificación binaria supervisada**. Sin embargo, el objetivo aplicado no consiste únicamente en asignar una etiqueta final de *impago* o *no impago*, sino en utilizar las probabilidades estimadas por el modelo como una puntuación de riesgo. Esta puntuación permite ordenar a los clientes según su riesgo relativo y analizar si los segmentos superiores del ranking concentran una mayor proporción de impagos.

#### 2.1.1. Regresión

En un problema de regresión, el objetivo es predecir una variable respuesta numérica continua a partir de un conjunto de variables explicativas. El caso más clásico es la regresión lineal, donde se asume que la respuesta puede describirse mediante una combinación

lineal más un término de error aleatorio [8]:

$$Y = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p + \varepsilon.$$

Si disponemos de  $n$  observaciones, el modelo puede escribirse en notación matricial como

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \mathbf{e},$$

donde  $\mathbf{X}$  representa la matriz de diseño,  $\boldsymbol{\beta}$  contiene el intercepto y los coeficientes asociados a las variables explicativas, y  $\mathbf{e}$  recoge los errores del modelo.

Desde la perspectiva de los Modelos Lineales Generalizados, la regresión lineal es un caso particular en el que la distribución condicional de la respuesta pertenece a la familia normal y la función de enlace es la identidad [10]:

$$\mu = \mathbb{E}(Y | X) = X\beta,$$

$$Y | X \sim \mathcal{N}(X\beta, \sigma^2 I).$$

El ajuste mediante mínimos cuadrados ordinarios busca encontrar el vector de parámetros que minimiza la suma de los errores al cuadrado:

$$J(\boldsymbol{\beta}) = \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2.$$

Siempre que  $\mathbf{X}^T \mathbf{X}$  sea invertible, la solución cerrada viene dada por

$$\boldsymbol{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}.$$

Aunque este trabajo se centra en una variable respuesta binaria, la regresión lineal resulta útil para introducir una idea básica: aproximar la respuesta esperada a partir de un conjunto de predictores. Esta idea sirve como punto de partida para entender la regresión logística, que adapta el enfoque lineal a problemas donde la variable objetivo toma valores 0 y 1. [10, 8].

### 2.1.2. Clasificación

En un problema de clasificación, el objetivo es predecir una variable respuesta categórica a partir de un conjunto de variables explicativas. En este trabajo interesa especialmente la clasificación binaria, donde la respuesta toma dos posibles valores, que habitualmente se codifican como 0 y 1 [8, 9].

Desde un punto de vista probabilístico, el objetivo no es solo asignar una clase, sino estimar la probabilidad de pertenecer a la clase positiva:

$$\mu = \mathbb{E}(Y | X) = \mathbb{P}(Y = 1 | X).$$

Esta igualdad es especialmente importante porque, cuando  $Y \in \{0, 1\}$ , la esperanza condicional coincide con la probabilidad de que ocurra el evento de interés. En el contexto

de este trabajo, dicho evento corresponde al impago.

## Regresión logística

Uno de los modelos más utilizados en clasificación binaria es la regresión logística. Aunque su nombre incluye el término “regresión”, su uso habitual se encuentra en problemas de clasificación, ya que permite modelar la probabilidad de que una observación pertenezca a la clase positiva [10, 8].

En el caso binario, el objetivo es modelar

$$\mu = \mathbb{P}(Y = 1 \mid X),$$

donde  $Y \in \{0, 1\}$ .

Esta probabilidad no resulta adecuada para representarse directamente mediante una combinación lineal de los predictores, ya que una expresión lineal podría tomar valores menores que 0 o mayores que 1.

La idea central de la regresión logística consiste en transformar la probabilidad mediante la función *logit*:

$$\text{logit}(\mu) = \log \left( \frac{\mu}{1 - \mu} \right).$$

El cociente

$$\frac{\mu}{1 - \mu}$$

recibe el nombre de *odds* o razón de probabilidades. La regresión logística asume que el logaritmo de dichas *odds* puede expresarse como una combinación lineal de los predictores [10]:

$$\log \left( \frac{\mu}{1 - \mu} \right) = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p.$$

Para una observación individual  $i$ , el predictor lineal se escribe como

$$\eta_i = \mathbf{x}_i^T \boldsymbol{\beta} = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip},$$

donde, por convenio,  $\mathbf{x}_i$  incluye un primer componente igual a 1 para incorporar el término independiente.

Formalmente, esta formulación se enmarca dentro de los Modelos Lineales Generalizados, al considerar una respuesta binaria modelada mediante una distribución Bernoulli y utilizar la función *logit* como función de enlace [10]. En particular, para cada observación se asume que

$$Y_i \mid \mathbf{x}_i \sim \text{Bernoulli}(\mu_i),$$

donde

$$\mu_i = \mathbb{P}(Y_i = 1 \mid \mathbf{x}_i).$$

La función de enlace establece que

$$\log\left(\frac{\mu_i}{1 - \mu_i}\right) = \mathbf{x}_i^T \boldsymbol{\beta}.$$

A partir de esta expresión se obtiene la función logística o sigmoide:

$$\mu_i = \frac{1}{1 + \exp(-\mathbf{x}_i^T \boldsymbol{\beta})}.$$

Esta transformación garantiza que las probabilidades estimadas se encuentren siempre entre 0 y 1. Por tanto, aunque el predictor lineal puede tomar cualquier valor real, la función logística lo transforma en una probabilidad válida.

A diferencia del modelo lineal clásico, en la regresión logística no se supone normalidad de los errores. La variabilidad de la respuesta viene determinada por la distribución Bernoulli:

$$\text{Var}(Y | X) = \mu(1 - \mu).$$

**Estimación del modelo** La estimación de la regresión logística se plantea como un problema de máxima verosimilitud. Para cada observación  $i$ , la función de probabilidad Bernoulli puede escribirse como [10, 8]

$$\mathbb{P}(Y_i = y_i | \mathbf{x}_i) = \mu_i^{y_i} (1 - \mu_i)^{1-y_i},$$

donde

$$\mu_i = \frac{1}{1 + \exp(-\mathbf{x}_i^T \boldsymbol{\beta})}.$$

Bajo el supuesto de independencia entre observaciones, la verosimilitud viene dada por

$$L(\boldsymbol{\beta}) = \prod_{i=1}^n \mu_i^{y_i} (1 - \mu_i)^{1-y_i}.$$

Resulta más conveniente trabajar con el logaritmo de la verosimilitud:

$$\ell(\boldsymbol{\beta}) = \sum_{i=1}^n [y_i \log(\mu_i) + (1 - y_i) \log(1 - \mu_i)].$$

Maximizar esta expresión es equivalente a minimizar su versión negativa, conocida como pérdida logarítmica o *log-loss*:

$$-\ell(\boldsymbol{\beta}) = -\sum_{i=1}^n [y_i \log(\mu_i) + (1 - y_i) \log(1 - \mu_i)].$$

**Regularización Lasso y Ridge** Cuando existen muchas variables explicativas o relaciones fuertes entre predictores, puede ser conveniente introducir regularización. La regularización añade una penalización a la función objetivo para controlar la magnitud de los coeficientes y reducir el riesgo de sobreajuste [9, 8].

En la penalización Ridge se añade una penalización basada en la norma  $L_2$  [11]:

$$\lambda \sum_{j=1}^p \beta_j^2.$$

Esta penalización reduce el tamaño de los coeficientes, aunque normalmente no los hace exactamente cero.

En la penalización Lasso se añade una penalización basada en la norma  $L_1$  [12]:

$$\lambda \sum_{j=1}^p |\beta_j|.$$

A diferencia de Ridge, Lasso puede llevar algunos coeficientes exactamente a cero, por lo que también puede actuar como un mecanismo de selección de variables.

En este trabajo se consideran estas variantes de la regresión logística porque permiten comparar un modelo lineal interpretable con versiones regularizadas, antes de pasar a modelos más flexibles como Random Forest o XGBoost.

## 2.2. Métricas de evaluación

Tener un modelo que produzca predicciones no es suficiente: también necesitamos una forma de evaluar su rendimiento. En clasificación binaria, las métricas se construyen comparando las predicciones del modelo con los valores reales de la variable respuesta [5, 8].

### 2.2.1. Accuracy, Precision, Recall y F1

En clasificación binaria, el comportamiento del modelo puede resumirse contando cuántas veces acierta y cuántas veces se equivoca para cada tipo de caso. La matriz de confusión recoge cuatro cantidades principales [5]:

- **TP (True Positives)**: casos positivos correctamente clasificados como positivos.
- **TN (True Negatives)**: casos negativos correctamente clasificados como negativos.
- **FP (False Positives)**: casos negativos clasificados erróneamente como positivos.
- **FN (False Negatives)**: casos positivos clasificados erróneamente como negativos.

A partir de estos valores se construyen distintas métricas de evaluación:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}, \quad Recall = \frac{TP}{TP + FN}, \quad Precision = \frac{TP}{TP + FP}.$$

La *accuracy* mide la proporción total de aciertos, pero puede ser engañosa cuando las clases están desbalanceadas. La *precision* indica qué proporción de los casos predichos como positivos son realmente positivos, mientras que el *recall* mide qué proporción de positivos reales son detectados por el modelo [3, 4].

Además, el *F1-score* combina *precision* y *recall* mediante su media armónica:

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

El uso de la media armónica hace que el *F1-score* penalice especialmente los casos en los que una de las dos métricas es baja. Por ello, esta medida solo alcanza valores elevados cuando el modelo mantiene simultáneamente una *precision* y un *recall* razonablemente altos.

En problemas como la predicción de impago, estas métricas son especialmente importantes porque la clase positiva suele ser minoritaria [3]. En este contexto, un *falso negativo* sería un cliente de alto riesgo que el modelo clasifica como de bajo riesgo, mientras que un *falso positivo* sería un cliente de bajo riesgo que el modelo considera erróneamente como de alto riesgo. Dependiendo de la estrategia de negocio, puede ser preferible priorizar la detección de impagos o reducir el número de falsas alarmas.

### 2.2.2. Curva ROC y ROC-AUC

Muchos modelos de clasificación no solo producen una clase final, sino también una probabilidad estimada de pertenencia a la clase positiva. A partir de estas probabilidades se puede variar el umbral de clasificación y estudiar cómo cambia el rendimiento del modelo.

La **curva ROC** representa la relación entre el **TPR** (*True Positive Rate*) y el **FPR** (*False Positive Rate*) para distintos valores del umbral. El TPR mide la proporción de positivos correctamente identificados, mientras que el FPR mide la proporción de negativos clasificados erróneamente como positivos [5].

El **ROC-AUC** resume en un solo número la capacidad del modelo para discriminar entre casos positivos y negativos. Un valor cercano a 1 indica alta capacidad discriminante, mientras que un valor cercano a 0,5 sugiere un comportamiento prácticamente aleatorio [5].

### 2.2.3. Curva Precision-Recall y PR-AUC

En problemas con clases desbalanceadas resulta especialmente útil analizar la **curva Precision-Recall**. Esta curva representa la relación entre la *precision* y el *recall* para distintos valores del umbral [6, 4].

A diferencia de la curva ROC, la curva Precision-Recall se centra directamente en la clase positiva. Esto es relevante en la predicción de impago, donde el evento de interés suele ser minoritario [4].

El **PR-AUC** resume el área bajo esta curva. Un valor más alto indica que el modelo consigue mantener un mejor equilibrio entre *precision* y *recall* a lo largo de distintos umbrales. Por este motivo, en contextos desbalanceados, el PR-AUC suele ser más informativo que la *accuracy* y complementa al ROC-AUC [6, 4].

En este trabajo, el ROC-AUC se utilizará para evaluar la capacidad general de discriminación, mientras que el PR-AUC será especialmente relevante para analizar el com-

portamiento del modelo sobre la clase minoritaria, es decir, los clientes que incurren en impago.

## 2.3. Desbalance de clases

En muchos problemas de clasificación binaria, las clases no aparecen representadas en la misma proporción. Esta situación se conoce como **desbalance de clases** y ocurre cuando una de las clases tiene muchos más casos que la otra [3].

En la predicción de impago, este problema es habitual, ya que la mayoría de los clientes cumplen con sus obligaciones de pago y solo una proporción menor incurre en impago. Por tanto, la clase positiva, asociada al evento de interés, suele estar formada por un número reducido de observaciones.

El desbalance puede afectar tanto al entrenamiento como a la evaluación de los modelos. Un modelo podría obtener una *accuracy* elevada simplemente clasificando a la mayoría de los clientes como no impago, pero esto no significaría que sea útil para detectar correctamente a los clientes de mayor riesgo [3, 4].

Por esta razón, en problemas desbalanceados resulta necesario analizar métricas adicionales a la *accuracy*. Métricas como el *recall*, la *precision*, el *F1-score*, el ROC-AUC y especialmente el PR-AUC permiten estudiar con mayor detalle el comportamiento del modelo sobre la clase minoritaria [6, 4].

Además, el desbalance también influye en la elección del umbral de clasificación. Aunque muchos modelos utilizan por defecto un umbral de 0,5, este valor no siempre es el más adecuado cuando la clase positiva es minoritaria. Dependiendo del objetivo del análisis, puede ser necesario ajustar el umbral para aumentar la detección de impagos o para reducir el número de falsas alarmas [3].

Existen distintas estrategias para tratar el desbalance de clases. Una opción consiste en modificar los pesos asignados a cada clase durante el entrenamiento, de forma que los errores cometidos sobre la clase minoritaria tengan mayor importancia. Otra posibilidad es aplicar técnicas de remuestreo, como el sobremuestreo de la clase minoritaria, el submuestreo de la clase mayoritaria o métodos sintéticos como SMOTE [3, 13].

En este trabajo, el desbalance de clases se considera un aspecto central del problema de predicción de impago. Por ello, la comparación de modelos no se basará únicamente en la *accuracy*, sino también en métricas que permitan evaluar la utilidad del modelo para identificar correctamente a los clientes con mayor probabilidad de impago.

## 2.4. Validación cruzada y Stratified K-Fold

Aunque una división simple entre entrenamiento y validación permite obtener una primera estimación del rendimiento del modelo, dicha estimación puede depender de la partición concreta realizada. La validación cruzada busca reducir esta dependencia repitiendo el proceso de entrenamiento y validación sobre distintas particiones del conjunto de datos [14, 8].

En la validación cruzada *k-fold*, el conjunto de datos se divide en *k* bloques o *folds*

aproximadamente del mismo tamaño. El modelo se entrena  $k$  veces, utilizando en cada repetición  $k - 1$  bloques para entrenamiento y el bloque restante para validación. Finalmente, se calcula la métrica de interés en cada repetición y se promedian los resultados obtenidos [14].

Por ejemplo, con  $k = 5$ , en cada repetición se utiliza aproximadamente el 80 % de los datos para entrenar y el 20 % restante para validar, cambiando el bloque de validación en cada iteración. De esta forma, cada observación participa una vez en la validación y se obtiene una estimación más estable del rendimiento del modelo.

En problemas de clasificación desbalanceada, como la predicción de impago, resulta especialmente conveniente utilizar validación cruzada estratificada, conocida habitualmente como *Stratified K-Fold*. Esta variante mantiene, en cada fold, una proporción similar de observaciones de cada clase. Así se evita que algún bloque de validación tenga una representación demasiado baja de la clase positiva, lo cual sería especialmente problemático cuando la clase de interés es minoritaria [14, 3].

En la parte experimental de este trabajo, la validación cruzada estratificada se utiliza como herramienta estándar para comparar de forma más justa los distintos modelos aplicados al problema de predicción de impago en el dataset *Home Credit Default Risk*. Para evitar fuga de información, las transformaciones dependientes de los datos, como la imputación de valores faltantes, la agrupación de categorías poco frecuentes, la codificación de variables categóricas, el escalado y las técnicas de remuestreo, se ajustan únicamente con los datos de entrenamiento de cada fold y se aplican después al fold de validación correspondiente.



# 3. Árboles de decisión y métodos de ensamblado

## 3.1. Árboles de decisión

Los árboles de decisión son modelos de aprendizaje supervisado que aproximan la relación entre las variables explicativas y la variable respuesta mediante particiones sucesivas del espacio de predictores. A diferencia de los modelos lineales, no imponen una forma funcional global, sino que dividen los datos en regiones y asignan una predicción a cada una de ellas [15, 8].

En este trabajo, los árboles de decisión se utilizan principalmente como base para modelos de ensamblado más robustos, como Random Forest y XGBoost. Su interés no está tanto en construir un único árbol interpretable, sino en entender la lógica sobre la que se apoyan modelos más potentes.

Sea  $X \in \mathbb{R}^p$  el vector de predictores y sea  $Y$  la variable respuesta. En clasificación binaria se considera

$$Y \in \{0, 1\},$$

donde el objetivo es aproximar la probabilidad condicional

$$p(x) = \mathbb{P}(Y = 1 \mid X = x).$$

La idea fundamental de un árbol consiste en dividir el espacio de predictores en un conjunto de regiones disjuntas,

$$\mathcal{R} = \{R_1, R_2, \dots, R_M\},$$

donde cada región corresponde a una hoja terminal. El modelo asigna una predicción constante a todas las observaciones que caen dentro de una misma región [15, 9]:

$$\hat{f}(x) = \sum_{m=1}^M c_m \mathbf{1}(x \in R_m).$$

En clasificación binaria, la probabilidad estimada de pertenecer a la clase positiva dentro de una hoja  $R_m$  se calcula como la proporción de observaciones positivas en dicha región:

$$\hat{p}_m = \frac{1}{N_m} \sum_{x_i \in R_m} y_i.$$

La construcción del árbol se realiza mediante partición binaria recursiva. En cada nodo se selecciona una variable  $X_j$  y un punto de corte  $s$ , generando una regla de decisión de

la forma

$$X_j < s.$$

Esta regla divide una región inicial  $R$  en dos nuevas regiones:

$$R_{\text{izq}}(j, s) = \{x \in R : x_j < s\},$$

$$R_{\text{der}}(j, s) = \{x \in R : x_j \geq s\}.$$

Para elegir las divisiones se utilizan medidas de impureza, que cuantifican el grado de mezcla de clases dentro de un nodo. Uno de los criterios más habituales es el índice de Gini [15]:

$$G(R_m) = 1 - \sum_{k=1}^K \hat{p}_{mk}^2.$$

Otra medida frecuente es la entropía [9, 8]:

$$H(R_m) = - \sum_{k=1}^K \hat{p}_{mk} \log(\hat{p}_{mk}).$$

En cada división, el algoritmo busca reducir la impureza del nodo original. Si una región  $R$  se divide en  $R_{\text{izq}}$  y  $R_{\text{der}}$ , la reducción de impureza puede escribirse como

$$\Delta I = I(R) - \left[ \frac{N_{\text{izq}}}{N} I(R_{\text{izq}}) + \frac{N_{\text{der}}}{N} I(R_{\text{der}}) \right],$$

donde  $I(\cdot)$  representa una medida de impureza, como Gini o entropía.

Los árboles son flexibles y pueden capturar relaciones no lineales e interacciones entre variables. Sin embargo, un árbol muy profundo puede adaptarse demasiado al conjunto de entrenamiento y perder capacidad de generalización. Por ello, se utilizan criterios de parada como la profundidad máxima, el número mínimo de observaciones por hoja o la mejora mínima exigida para aceptar una nueva división [15, 8].

En resumen, un árbol de decisión puede entenderse como una secuencia de reglas que divide los datos en grupos cada vez más homogéneos. Esta idea será la base de los métodos de ensamblado utilizados posteriormente.

## 3.2. Bagging

Una limitación importante de los árboles de decisión es su alta varianza: pequeñas variaciones en los datos pueden generar árboles bastante diferentes. Para reducir esta inestabilidad se utilizan métodos de ensamblado, que combinan varios modelos base en lugar de depender de un único modelo [9, 8].

Una de las estrategias más importantes es el *Bootstrap Aggregating*, conocido como *bagging*. La idea consiste en entrenar varios árboles sobre distintas muestras bootstrap del conjunto de datos y combinar posteriormente sus predicciones [16].

Sea

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$$

el conjunto de datos original. El bagging genera  $B$  muestras bootstrap,

$$\mathcal{D}^{(1)}, \mathcal{D}^{(2)}, \dots, \mathcal{D}^{(B)},$$

obtenidas mediante muestreo con reemplazo de las observaciones de  $\mathcal{D}$ , por lo que algunas observaciones pueden repetirse y otras quedar fuera en cada muestra. A continuación, sobre cada muestra se ajusta un modelo base:

$$\hat{f}^{(b)}(x), \quad b = 1, \dots, B.$$

En clasificación, la predicción final puede obtenerse mediante votación mayoritaria:

$$\hat{f}_{\text{bag}}(x) = \text{mode} \left\{ \hat{f}^{(1)}(x), \dots, \hat{f}^{(B)}(x) \right\}.$$

Si los modelos base proporcionan probabilidades estimadas, también puede promediarse la probabilidad predicha. En el caso binario:

$$\hat{p}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{p}^{(b)}(x).$$

Desde una perspectiva intuitiva, el bagging estabiliza el modelo: en lugar de confiar en un único árbol, combina muchos árboles entrenados sobre pequeñas variaciones del conjunto de datos. Su principal efecto es reducir la varianza, especialmente cuando los modelos base son árboles de decisión [16, 9].

### 3.3. Random Forest

Random Forest es una extensión del bagging aplicada a árboles de decisión. Al igual que en bagging, se generan múltiples muestras bootstrap y se ajusta un árbol sobre cada una de ellas. La diferencia principal es que, en cada nodo del árbol, la mejor división se busca solo dentro de un subconjunto aleatorio de variables explicativas [17].

Esta aleatoriedad adicional busca reducir la correlación entre los árboles. Si todos los árboles consideran siempre todas las variables, es probable que utilicen los mismos predictores dominantes en las primeras divisiones. Random Forest evita parcialmente este problema obligando a cada árbol a explorar diferentes subconjuntos de variables [17, 9].

Sea

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$$

el conjunto de datos original, donde  $x_i \in \mathbb{R}^p$  representa el vector de variables explicativas de la observación  $i$ , siendo  $p$  el número total de predictores.

A partir de este conjunto, el algoritmo construye  $B$  árboles. Para cada  $b = 1, \dots, B$ , se genera una muestra bootstrap  $\mathcal{D}^{(b)}$  y se ajusta un árbol de decisión  $\hat{f}^{(b)}(x)$ . A diferencia

del bagging clásico, en cada nodo no se consideran todos los predictores, sino que se selecciona aleatoriamente un subconjunto de tamaño  $m$ , con

$$m < p.$$

La mejor división se busca únicamente dentro de ese subconjunto de variables. Este procedimiento se repite de forma independiente en cada nodo y en cada árbol.

En problemas de clasificación, la predicción final puede obtenerse mediante votación mayoritaria:

$$\hat{f}_{\text{RF}}(x) = \text{mode} \left\{ \hat{f}^{(1)}(x), \dots, \hat{f}^{(B)}(x) \right\}.$$

Si se trabaja con probabilidades de clase, estas también pueden agregarse. En clasificación binaria, si  $\hat{p}^{(b)}(x)$  representa la probabilidad estimada por el árbol  $b$  para la clase positiva, entonces:

$$\hat{p}_{\text{RF}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{p}^{(b)}(x).$$

Posteriormente, la clase predicha puede obtenerse aplicando un umbral de decisión sobre  $\hat{p}_{\text{RF}}(x)$ .

Random Forest no requiere asumir linealidad, normalidad de los errores ni una forma funcional concreta entre los predictores y la respuesta. Además, puede capturar relaciones no lineales e interacciones entre variables de forma automática. Su principal ventaja es que suele ofrecer un buen rendimiento predictivo y mayor estabilidad que un árbol individual [17, 8].

No obstante, esta mejora predictiva viene acompañada de una menor interpretabilidad. Mientras que un árbol pequeño puede representarse como una secuencia clara de reglas, un Random Forest combina muchos árboles, por lo que resulta más difícil explicar cada predicción de forma directa. Aun así, pueden analizarse medidas de importancia de variables para identificar qué predictores contribuyen más al rendimiento del modelo [17, 9].

En conjunto, Random Forest puede entenderse como una evolución natural de los árboles de decisión: mantiene su flexibilidad, pero reduce su inestabilidad mediante la combinación de muchos árboles y la introducción de aleatoriedad en la selección de variables.

## 4. Boosting y XGBoost

Los métodos de *boosting* constituyen una familia de modelos de ensamblado dentro del aprendizaje supervisado. Al igual que el *bagging* y Random Forest, combinan varios modelos base. Sin embargo, la diferencia principal está en la forma de construirlos: mientras que en el *bagging* los modelos se entrenan de forma independiente, en el *boosting* se construyen de manera secuencial [18, 9].

La idea central es sencilla: cada nuevo modelo intenta corregir los errores que todavía comete el modelo acumulado. Por ello, el modelo final no depende de un único árbol, sino de la suma de muchos modelos simples, normalmente árboles de baja profundidad [18, 8].

En este trabajo, el interés principal se centra en XGBoost, una implementación eficiente y regularizada del *Gradient Boosting*. Este modelo resulta especialmente adecuado para datos tabulares, ya que puede capturar relaciones no lineales e interacciones entre variables, incorporando además mecanismos para controlar el sobreajuste [19].

### 4.1. Idea general del boosting

De forma general, un modelo de *boosting* puede escribirse como una suma de modelos base:

$$f_M(x) = \sum_{m=1}^M \alpha_m h_m(x),$$

donde  $h_m(x)$  representa el modelo base añadido en la iteración  $m$ ,  $\alpha_m$  es su contribución al modelo final y  $M$  es el número total de iteraciones [18, 9].

Esta formulación muestra que el *boosting* no ajusta un único modelo complejo desde el inicio, sino que lo construye poco a poco. En cada etapa, se añade un nuevo componente que mejora el modelo acumulado hasta ese momento.

La diferencia respecto al *bagging* es importante. En *bagging*, los árboles se entrenan en paralelo y luego se agregan. En *boosting*, cada árbol depende del modelo anterior. Por tanto, el aprendizaje es secuencial: las iteraciones posteriores se concentran en los errores que aún no han sido corregidos [9, 8].

### 4.2. Gradient Boosting

El *Gradient Boosting* interpreta el *boosting* como un problema de optimización. El objetivo consiste en encontrar una función  $f$  que minimice una pérdida empírica, es decir, una medida del error calculada sobre las observaciones disponibles en el conjunto de entrenamiento [18]:

$$\min_f \sum_{i=1}^n L(y_i, f(x_i)).$$

Aquí,  $L(\cdot)$  mide el error entre el valor observado  $y_i$  y la predicción del modelo  $f(x_i)$ .

El modelo se construye de forma aditiva:

$$f_M(x) = f_0(x) + \sum_{m=1}^M \gamma_m h_m(x),$$

donde  $f_0(x)$  es una predicción inicial,  $h_m(x)$  es el modelo base añadido en la iteración  $m$ , y  $\gamma_m$  controla la contribución de dicho modelo.

La predicción inicial suele definirse como una constante. Para ello, se consideran posibles valores constantes  $c$  y se elige aquel que minimiza la pérdida empírica:

$$c^* = \arg \min_c \sum_{i=1}^n L(y_i, c).$$

Aquí,  $c$  representa una constante candidata, mientras que  $c^*$  es la constante óptima obtenida al resolver dicha minimización. De este modo, el modelo inicial queda dado por

$$f_0(x) = c^*.$$

A partir de esta predicción inicial, el modelo se mejora de manera iterativa. En cada iteración se calculan los pseudo-residuos, definidos como el gradiente negativo de la función de pérdida evaluado en el modelo actual:

$$r_{im} = - \left[ \frac{\partial L(y_i, F)}{\partial F} \right]_{F=f_{m-1}(x_i)}.$$

Estos pseudo-residuos indican en qué dirección debe corregirse la predicción para reducir la pérdida. Por tanto, el nuevo árbol no se ajusta directamente sobre la variable respuesta original, sino sobre los errores o direcciones de corrección que aún quedan en el modelo acumulado [18].

Después de ajustar el modelo base  $h_m(x)$ , el modelo se actualiza como

$$f_m(x) = f_{m-1}(x) + \gamma_m h_m(x).$$

Una forma habitual de controlar el aprendizaje consiste en introducir una tasa de aprendizaje  $\eta$ :

$$f_m(x) = f_{m-1}(x) + \eta \gamma_m h_m(x), \quad 0 < \eta \leq 1.$$

Valores pequeños de  $\eta$  hacen que el aprendizaje sea más gradual y suelen mejorar la generalización, aunque requieren más iteraciones. En cambio, valores grandes aceleran el ajuste, pero pueden aumentar el riesgo de sobreajuste [18, 9].

En conjunto, el *Gradient Boosting* puede entenderse como una forma de construir un modelo flexible mediante correcciones sucesivas. Su principal fortaleza es que permite aproximar relaciones complejas, pero requiere controlar adecuadamente el número de árboles, la tasa de aprendizaje y la complejidad de cada árbol.

### 4.3. XGBoost

XGBoost, abreviatura de *eXtreme Gradient Boosting*, es una implementación eficiente y regularizada del *Gradient Boosting*. Su objetivo es mejorar la capacidad predictiva y la estabilidad del modelo mediante árboles de decisión, regularización explícita e información de segundo orden durante la optimización [19].

En XGBoost, el modelo se construye como una suma de árboles. Para cada observación  $x_i$ , la predicción en la iteración  $t$  puede escribirse como

$$\hat{y}_i^{(t)} = \sum_{k=1}^t f_k(x_i), \quad f_k \in \mathcal{F},$$

donde  $\mathcal{F}$  representa el espacio de árboles de regresión. De forma equivalente, la actualización en la iteración  $t$  se expresa como

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(x_i).$$

Esto refleja la misma lógica del *boosting*: cada nuevo árbol se añade para corregir la predicción acumulada del modelo anterior.

La función objetivo de XGBoost combina una función de pérdida y un término de regularización [19]:

$$\mathcal{L} = \sum_{i=1}^n L(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k).$$

Para un árbol  $f$ , el término de regularización puede escribirse como

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2,$$

donde  $T$  es el número de hojas del árbol,  $w_j$  es el valor asociado a la hoja  $j$ ,  $\gamma$  penaliza el número de hojas y  $\lambda$  penaliza la magnitud de los pesos de las hojas.

En la iteración  $t$ , el objetivo consiste en encontrar un nuevo árbol  $f_t$  que mejore la predicción actual:

$$\mathcal{L}^{(t)} = \sum_{i=1}^n L(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t).$$

Para optimizar esta expresión, XGBoost utiliza una aproximación de Taylor de segundo orden alrededor de la predicción actual [19]:

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^n \left[ g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2 \right] + \Omega(f_t),$$

donde

$$g_i = \frac{\partial L(y_i, \hat{y}_i^{(t-1)})}{\partial \hat{y}_i^{(t-1)}}, \quad h_i = \frac{\partial^2 L(y_i, \hat{y}_i^{(t-1)})}{\partial (\hat{y}_i^{(t-1)})^2}.$$

Aquí  $g_i$  representa el gradiente y  $h_i$  la segunda derivada de la pérdida respecto a la

predicción actual. Esta es una diferencia importante frente al *Gradient Boosting* básico, ya que XGBoost utiliza información de primer y segundo orden para decidir cómo corregir el modelo.

Cada árbol divide el espacio de predictores en hojas. Si una observación  $x_i$  cae en la hoja  $j$ , entonces

$$f_t(x_i) = w_j.$$

Sea  $I_j$  el conjunto de observaciones que caen en la hoja  $j$ . Se definen

$$G_j = \sum_{i \in I_j} g_i, \quad H_j = \sum_{i \in I_j} h_i.$$

Para una estructura de árbol fija, el valor óptimo del peso de la hoja  $j$  viene dado por

$$w_j^* = -\frac{G_j}{H_j + \lambda}.$$

Esta fórmula muestra que el valor asignado a cada hoja depende de la suma de gradientes y segundas derivadas de las observaciones que contiene, moderada por la regularización  $\lambda$ .

Para evaluar una posible división de un nodo en dos hijos, izquierdo y derecho, XGBoost utiliza la ganancia [19]:

$$\text{Gain} = \frac{1}{2} \left[ \frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G^2}{H + \lambda} \right] - \gamma.$$

Esta ganancia mide cuánto mejora la función objetivo al realizar una división. Si la mejora no compensa la penalización asociada a crear nuevas hojas, la división puede no ser aceptada.

Desde una perspectiva intuitiva, XGBoost construye muchos árboles de forma secuencial. Cada árbol no intenta explicar por completo la variable respuesta, sino corregir lo que el modelo acumulado todavía no explica bien. Además, la regularización ayuda a evitar que el modelo crezca de forma excesiva y se adapte demasiado al conjunto de entrenamiento.

## 4.4. Parámetros principales y control del sobreajuste

El rendimiento de XGBoost depende de la elección de sus hiperparámetros. Estos no se estiman directamente como los coeficientes de una regresión, sino que se fijan antes del entrenamiento y se seleccionan mediante validación [19, 9].

Uno de los parámetros más importantes es la tasa de aprendizaje  $\eta$ , que controla cuánto aporta cada nuevo árbol al modelo acumulado. También es importante el número de árboles, ya que existe una relación directa entre ambos: una tasa de aprendizaje pequeña suele requerir más árboles, mientras que una tasa más alta puede ajustar más rápido, pero con mayor riesgo de sobreajuste [18, 19].

La profundidad máxima de los árboles, normalmente indicada como `max_depth`, controla la complejidad de cada árbol. Árboles poco profundos capturan relaciones más simples,



mientras que árboles más profundos permiten representar interacciones más complejas, aunque también pueden sobreajustar.

Los parámetros de regularización también son relevantes. El parámetro  $\lambda$  penaliza la magnitud de los pesos de las hojas, mientras que  $\gamma$  exige una ganancia mínima para aceptar una nueva división. Valores más altos generan modelos más conservadores [19].

Además, XGBoost permite introducir aleatoriedad mediante el muestreo de observaciones y variables. El parámetro `subsample` controla la proporción de observaciones utilizadas en cada iteración, mientras que `colsample_bytree` controla la proporción de variables consideradas. Estos mecanismos ayudan a mejorar la generalización [19, 20].

En problemas de clasificación desbalanceada, como la predicción de impago, también puede ajustarse el parámetro `scale_pos_weight`, que asigna mayor peso a la clase positiva. Esto puede ser útil cuando el objetivo es mejorar la detección de la clase minoritaria [3, 19].

En conjunto, XGBoost busca equilibrar capacidad de ajuste y capacidad de generalización. Un modelo con demasiados árboles, profundidad elevada y poca regularización puede sobreajustar. En cambio, un modelo demasiado restringido puede no capturar suficientemente la estructura de los datos. Por ello, en este trabajo se evalúan distintas configuraciones de XGBoost mediante validación cruzada, prestando especial atención al PR-AUC y al comportamiento del modelo sobre la clase minoritaria.

## 5. Aplicación práctica: Predicción y scoring de riesgo de impago

### 5.1. Introducción al dataset Home Credit Default Risk

En esta aplicación práctica se trabaja con el conjunto de datos *Home Credit Default Risk*, procedente de una competición de Kaggle organizada por Home Credit Group. El objetivo es estimar la probabilidad de que un cliente presente dificultades de pago, por lo que el problema se formula como una tarea de clasificación binaria supervisada. La variable objetivo **TARGET** toma el valor 1 cuando el cliente presenta dificultades de pago y 0 en el resto de casos [7].

En el conjunto de datos analizado, la clase positiva representa aproximadamente el 8,07% del total de observaciones, mientras que el 91,93% restante corresponde a clientes sin dificultades de pago. Esta distribución confirma el fuerte desbalanceo de clases y justifica que la evaluación no se base únicamente en la *accuracy*, ya que un modelo podría obtener una exactitud elevada clasificando mayoritariamente a los clientes como no impago.

Desde el punto de vista aplicado, el interés del trabajo no reside solo en asignar una etiqueta final a cada cliente, sino en estudiar si las probabilidades estimadas pueden utilizarse como una puntuación de riesgo. En este sentido, el modelo se interpreta como una herramienta de *scoring*, orientada a ordenar las solicitudes según su riesgo relativo de impago y a apoyar procesos de priorización crediticia [1, 2, 21].

Por ello, además de métricas habituales de clasificación como *precision*, *recall*, F1, ROC-AUC y PR-AUC, se analiza la capacidad del modelo para concentrar clientes con impago en los segmentos de mayor riesgo. Este enfoque permite valorar si el score generado puede ser útil para priorizar revisiones, segmentar solicitudes y asignar recursos de análisis a los casos con mayor probabilidad estimada de impago [3, 6, 4].

El análisis se construye a partir de la tabla principal `application_train.csv`, que contiene la variable objetivo y la información general de cada solicitud. Posteriormente, se incorpora información histórica procedente de las tablas `bureau.csv` y `previous_application.csv`, con el objetivo de enriquecer la representación de cada solicitante mediante variables agregadas sobre su comportamiento crediticio previo. La Tabla 5.1 resume las fuentes de datos utilizadas y el tratamiento aplicado a cada una de ellas.

### 5.2. Preparación y exploración de datos

La tabla `application_train` se utilizó como base principal del modelado, al contener una única observación por solicitud y la variable objetivo **TARGET**. Las tablas `bureau` y `previous_application` se trataron como fuentes auxiliares, ya que aportan información

histórica adicional, pero presentan múltiples registros por cliente y no están disponibles necesariamente para todos los solicitantes [7].

Tabla 5.1: Resumen de las fuentes de datos utilizadas

| Fuente                            | Descripción y tratamiento                                                                                                                                                                                                                                                                              |
|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>application_train</code>    | Tabla principal del problema. Contiene una observación por solicitud/cliente, la variable objetivo <b>TARGET</b> y la información de la solicitud actual. A partir de esta tabla se construyeron variables temporales, ratios financieros y agrupaciones de algunas variables discretas y categóricas. |
| <code>bureau</code>               | Tabla auxiliar con múltiples registros de créditos históricos externos por cliente. Se agregó a nivel de <code>SK_ID_CURR</code> para resumir el historial crediticio externo mediante variables como número de créditos, créditos activos, créditos cerrados, deuda, límites y retrasos.              |
| <code>previous_application</code> | Tabla auxiliar con múltiples solicitudes previas por cliente. Se agregó a nivel de <code>SK_ID_CURR</code> para resumir información sobre solicitudes anteriores, aprobaciones, rechazos, importes medios, antigüedad de la última solicitud y tasa de aprobación histórica.                           |

*Nota.* Las variables agregadas de `bureau` y `previous_application` se incorporaron al conjunto principal mediante *left join*, manteniendo una única fila por solicitud/cliente.

En una primera etapa se revisó la estructura de las tablas, la presencia de valores ausentes, posibles duplicados y la relación entre las distintas fuentes de información. Esta exploración permitió confirmar que las tablas auxiliares debían agregarse previamente a nivel de cliente mediante la clave `SK_ID_CURR`, antes de ser incorporadas al conjunto principal de modelado.

Una vez construidas las variables agregadas, estas se añadieron a `application_train` mediante un *left join*. Este criterio permitió conservar todas las solicitudes de la tabla principal, incluso cuando un cliente no disponía de historial en alguna de las fuentes auxiliares, manteniendo una única fila por solicitud.

Después de esta integración, se separó el conjunto de entrenamiento del conjunto de test. A partir de ese punto, las transformaciones posteriores se aplicaron de forma separada sobre ambos conjuntos, con el fin de mantener el test como conjunto final de evaluación y evitar que influyera en las decisiones de preparación y modelado.

Dentro de estas transformaciones, la ausencia de registros en las tablas auxiliares no se trató únicamente como un valor faltante, sino también como una posible señal de ausencia de historial crediticio. Por este motivo, antes de la imputación se construyeron indicadores binarios como `has_bureau_history` y `has_previous_application_history`. Asimismo, las variables de conteo asociadas al historial se completaron con cero cuando la ausencia de información representaba la inexistencia de registros previos, mientras que otros valores

ausentes se reservaron para ser tratados posteriormente durante el preprocesamiento.

Además de la información histórica incorporada, se generaron variables derivadas tanto a partir de la tabla principal como de las variables agregadas procedentes de las tablas auxiliares. Estas transformaciones incluyeron conversiones de variables temporales expresadas en días, ratios financieros, resúmenes de fuentes externas de riesgo, variables relacionadas con consultas al buró e indicadores asociados al historial crediticio del cliente. El objetivo de esta etapa fue obtener variables más interpretables y útiles para capturar diferencias de riesgo entre solicitudes. También se realizaron agrupamientos manuales de algunas variables de conteo y de ciertas variables categóricas, generando nuevas variables con sufijo `_GROUP`. Esta decisión permitió reducir una granularidad excesiva, controlar categorías o valores poco frecuentes y facilitar la interpretación de los patrones aprendidos por los modelos. Por ejemplo, algunos conteos de créditos, solicitudes previas o consultas al buró se transformaron en intervalos como 0, 1, 2-3 o 4+, según el caso.

Finalmente, antes de la comparación de modelos, se realizó una limpieza de calidad sobre el conjunto de variables candidatas. En esta etapa se revisaron variables con baja variabilidad, relaciones redundantes y posibles solapamientos derivados de las transformaciones efectuadas. Cuando dos variables aportaban información muy similar, se priorizó aquella con una interpretación más clara desde el punto de vista del riesgo crediticio.

Tras completar la integración, transformación, agrupamiento manual y limpieza final, el conjunto de modelado quedó formado por una única observación por solicitud, la variable objetivo `TARGET` y 62 variables predictoras antes de la codificación de las variables categóricas. Este conjunto constituye la base de los experimentos de modelado desarrollados en las secciones posteriores.

### 5.3. Diseño experimental y estrategia de evaluación

Una vez construido el conjunto de datos integrado y depurado, se definió una estrategia experimental para comparar modelos y evaluar su utilidad en un problema de clasificación binaria fuertemente desbalanceado. El objetivo no fue únicamente maximizar la exactitud global, sino analizar la capacidad del modelo para identificar, ordenar y priorizar correctamente los casos de mayor riesgo [3, 4].

A partir del conjunto de datos preparado, se generó una partición estratificada para separar el proceso de desarrollo del modelo de la evaluación final. En concreto, se seleccionó una muestra de 40,000 observaciones como conjunto de desarrollo, manteniendo aproximadamente la misma proporción de clientes con y sin impago que en el conjunto original. Las 267,511 observaciones restantes se reservaron como conjunto de test final y no participaron en la comparación de modelos, selección de variables, tratamiento del desbalanceo ni ajuste de configuraciones.

El conjunto de desarrollo quedó compuesto por 40,000 observaciones y 62 variables predictoras antes de la codificación de variables categóricas. La comparación entre modelos se realizó mediante validación cruzada estratificada con cinco particiones (`StratifiedKFold`), mezcla aleatoria y `random_state = 42`. Este procedimiento permitió preservar la distribución de la variable objetivo en cada fold y obtener una estimación más estable del

rendimiento de los modelos [14, 3].

Todas las etapas de preprocesamiento utilizadas en la validación cruzada se ajustaron dentro de cada fold empleando únicamente el subconjunto de entrenamiento correspondiente. En las variables numéricas, cuando existían valores ausentes en el entrenamiento, se incorporó un indicador binario de ausencia y posteriormente se imputaron los valores faltantes mediante la mediana calculada en dicho subconjunto. En el caso de las variables categóricas, los valores ausentes se sustituyeron por la categoría **Missing**. Además, en las variables categóricas no agrupadas manualmente, las categorías con una frecuencia inferior al 1 % en el fold de entrenamiento se agruparon bajo la categoría **Other**. Este diseño evita que la información del fold de validación influya en las transformaciones realizadas durante el entrenamiento [22].

Posteriormente, las variables categóricas fueron transformadas mediante codificación *one-hot*. Dado que esta transformación se ajustó de forma independiente en cada fold, el número final de variables utilizadas por los modelos varió ligeramente entre particiones, situándose entre 167 y 169 variables, con un promedio de 168,4. Finalmente, las columnas de entrenamiento y validación se alinearon para garantizar que ambos conjuntos utilizaran la misma representación.

El flujo experimental se planteó de forma progresiva. En primer lugar, se compararon distintos modelos con el objetivo de establecer una referencia inicial de rendimiento bajo un esquema común de validación. En esta etapa se evaluaron modelos lineales, como la regresión logística y la regresión logística con regularización, junto con modelos basados en árboles, como Random Forest y XGBoost [17, 19]. Dado el fuerte desbalanceo de la variable objetivo, esta comparación inicial ya incorporó mecanismos básicos de ponderación de clases.

En los modelos implementados en scikit-learn que permiten ponderación de clases, como la regresión logística y Random Forest, se utilizó `class_weight="balanced"`, que pondera las clases de forma inversamente proporcional a su frecuencia [23, 24]. En XGBoost, donde este parámetro no se define mediante `class_weight`, se empleó `scale_pos_weight`, calculado en cada fold como la razón entre observaciones negativas y positivas del conjunto de entrenamiento [25].

A partir de esta comparación inicial, el análisis se centró en XGBoost, al tratarse de un modelo competitivo para datos tabulares y capaz de capturar relaciones no lineales e interacciones entre variables. Sobre este modelo candidato se evaluaron estrategias adicionales frente al desbalanceo, incluyendo distintas intensidades de ponderación de la clase positiva, sobremuestreo aleatorio, submuestreo aleatorio y SMOTE. Dentro de este análisis también se probó una versión moderada de la ponderación de clases, utilizando el 75 % del valor base de `scale_pos_weight` calculado en cada fold [19, 13, 3].

Las métricas utilizadas fueron *accuracy*, *precision*, *recall*, F1, ROC-AUC y PR-AUC. La *accuracy* se reportó únicamente como referencia, ya que puede resultar engañosa cuando la clase mayoritaria domina el conjunto de datos. En cambio, PR-AUC se consideró la métrica principal de comparación, porque evalúa el compromiso entre *precision* y *recall*, lo que permite analizar la calidad de las alertas sobre la clase positiva en un contexto de

fuerte desbalanceo [6, 4].

ROC-AUC se utilizó como métrica complementaria para evaluar la capacidad general de discriminación del modelo, es decir, su capacidad para asignar mayores puntuaciones de riesgo a clientes que realmente incurren en impago frente a clientes que no lo hacen. Por su parte, *precision*, *recall* y F1 se interpretaron principalmente en el análisis de umbrales, ya que dependen del punto de corte utilizado para transformar una probabilidad en una clasificación final [5].

Dado que el objetivo aplicado del trabajo es evaluar el modelo como herramienta de scoring, las métricas globales se complementaron con análisis de deciles de riesgo, *lift* y captura acumulada de impagos. Estas medidas permiten comprobar si las probabilidades estimadas por el modelo concentran una mayor proporción de impagos en los segmentos superiores del ranking, aspecto clave para valorar su utilidad en procesos de revisión crediticia [1, 21].

En consecuencia, la selección del modelo candidato no se basó únicamente en una métrica aislada. Primero se utilizaron PR-AUC y ROC-AUC para identificar modelos con capacidad predictiva razonable. Posteriormente, cuando varias alternativas presentaron valores cercanos de PR-AUC, se consideró también el equilibrio entre *precision*, *recall* y F1. Finalmente, la utilidad práctica del ranking se evaluó mediante deciles, *lift* y captura acumulada, con el fin de determinar si el score generado podía servir para priorizar clientes de mayor riesgo.

La utilización de un conjunto de test independiente permitió que las decisiones metodológicas principales se realizaran exclusivamente sobre el conjunto de desarrollo. El conjunto de test se reservó para la evaluación final del modelo seleccionado, aplicando las transformaciones aprendidas a partir del conjunto de entrenamiento final. De esta forma, se redujo el riesgo de sobreajuste y se obtuvo una estimación más realista del rendimiento esperado sobre datos no utilizados durante el proceso de selección y ajuste del modelo [14, 22].

## 5.4. Interpretación del desbalanceo

El fuerte desbalanceo de clases condiciona directamente la interpretación de los resultados. En el conjunto analizado, la clase positiva representa una proporción reducida del total, por lo que incluso un modelo con capacidad predictiva razonable puede obtener valores modestos de *precision*. Esto ocurre porque, aunque el modelo consiga identificar clientes con mayor riesgo, el número absoluto de clientes sin impago sigue siendo mucho mayor [3, 4].

Tabla 5.2: Distribución de clases de la variable objetivo

| Clase | Significado | N.º observaciones | Porcentaje (%) |
|-------|-------------|-------------------|----------------|
| 0     | Sin impago  | 282 686           | 91.93          |
| 1     | Con impago  | 24 825            | 8.07           |

Por esta razón, existe un compromiso natural entre *precision* y *recall*. Si se utiliza un umbral bajo, el modelo clasifica más clientes como positivos y aumenta el *recall*, pero reduce la *precision* al incorporar más falsos positivos. En cambio, si se utiliza un umbral más alto, el modelo es más selectivo y puede mejorar la *precision*, aunque a costa de capturar menos casos positivos [6, 4].

En este contexto, la elección del umbral no debe verse como una decisión puramente técnica. Depende también del coste relativo de los falsos positivos y falsos negativos. En una aplicación real, detectar más clientes de riesgo puede ser útil si el coste de revisar falsos positivos es aceptable. Por el contrario, si la capacidad de revisión es limitada, puede preferirse un umbral más exigente que priorice una mayor precisión [1, 21].

Por ello, el modelo no se interpreta como una herramienta automática de aprobación o rechazo, sino como un mecanismo de apoyo para ordenar clientes según su riesgo relativo. Antes de fijar un umbral de clasificación definitivo, resulta necesario seleccionar una configuración que ofrezca un equilibrio razonable entre capacidad predictiva, estabilidad y utilidad práctica.

## 5.5. Enfoque aplicado del scoring y valor diferencial del análisis

Como se ha comentado en los capítulos anteriores, el conjunto de datos *Home Credit Default Risk* ha sido ampliamente utilizado en competiciones y trabajos de investigación orientados al desarrollo de modelos predictivos. En muchos de estos trabajos, el objetivo principal consiste en maximizar métricas globales de rendimiento, como ROC-AUC, o mejorar la posición obtenida en competiciones tipo *leaderboard* [7].

En este trabajo se adopta un enfoque diferente. El interés principal no reside únicamente en desarrollar un modelo con buen rendimiento predictivo, sino en analizar si el *score* generado puede utilizarse como herramienta de apoyo para priorizar clientes con mayor riesgo de impago en un contexto operativo [1, 2, 21].

Por este motivo, además de las métricas tradicionales de clasificación, el análisis incorpora indicadores propios de modelos de *scoring*, como deciles de riesgo, *lift*, captura acumulada y análisis de umbrales. Estos indicadores permiten evaluar si el modelo consigue concentrar una mayor proporción de impagos en los primeros segmentos del ranking y, por tanto, si el *score* resulta útil para orientar procesos de revisión crediticia [21].

La Tabla 5.3 resume las principales diferencias entre el enfoque habitual encontrado en trabajos previos y el enfoque adoptado en este TFM.

Tabla 5.3: Comparación entre enfoques habituales encontrados y el enfoque adoptado en este TFM.

| Aspecto                               | Enfoques habituales encontrados                                                                                                             | Enfoque de este TFM                                                                                               |
|---------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Objetivo principal</b>             | Maximizar la capacidad discriminante del modelo, normalmente mediante el ROC-AUC utilizado en el <i>leaderboard</i> .                       | Evaluar la utilidad práctica del <i>score</i> para priorizar riesgo.                                              |
| <b>Uso del <i>score</i></b>           | Probabilidad predicha de <i>default</i> /TARGET, utilizada principalmente para ordenar clientes y generar la predicción de competición.     | <i>Ranking</i> operativo para revisión, segmentación y apoyo al analista.                                         |
| <b>Métricas</b>                       | ROC-AUC como métrica oficial de competición; otras métricas globales pueden aparecer como análisis complementario.                          | PR-AUC, ROC-AUC, deciles, <i>lift</i> , <i>top-k</i> , umbrales y equilibrio <i>precision/recall</i> .            |
| <b>Interpretación del desbalanceo</b> | Se reconoce como un problema técnico del modelado, asociado al bajo peso relativo de la clase de impago.                                    | Se interpreta además como una limitación operativa que condiciona el uso del modelo y la lectura de sus métricas. |
| <b>Uso de umbrales</b>                | La evaluación oficial no exige fijar un umbral de clasificación; cuando se reporta clasificación, suele usarse un umbral fijo o secundario. | Se analizan distintos umbrales según el equilibrio <i>precision/recall</i> requerido.                             |
| <b>Valor de negocio</b>               | Mejor capacidad discriminante medida por una métrica de competición.                                                                        | Priorización de recursos: decidir qué clientes revisar primero y con qué intensidad.                              |
| <b>Decisión crediticia</b>            | No suele formalizarse una regla de aprobación/rechazo; el foco está en la predicción y comparación de modelos.                              | Se plantea como apoyo a la decisión, no como sustitución de la decisión final.                                    |
| <b>Diferenciación metodológica</b>    | Modelos, ensamblados y <i>feature engineering</i> orientados a mejorar el rendimiento en el <i>leaderboard</i> .                            | Evaluación crítica del <i>score</i> mediante deciles, <i>lift</i> , <i>top-k</i> y umbrales operativos.           |

*Nota:* elaboración propia. El *leaderboard* de Kaggle clasifica los modelos según la métrica oficial de la competición, en este caso el ROC-AUC. En este TFM, la comparación se orienta a destacar un enfoque más operativo del *score*, basado en ranking, deciles, *lift*, *top-k* y umbrales.

Como consecuencia, el modelo desarrollado no se plantea como un mecanismo automático de aprobación o rechazo de solicitudes de crédito. Su finalidad es servir como herramienta de apoyo para priorizar revisiones, optimizar recursos y facilitar la toma de decisiones por parte del analista de riesgo.



## 5.6. Comparación inicial de modelos mediante PR-AUC y ROC-AUC

Como punto de partida se realizó una comparación entre distintos modelos de clasificación con el objetivo de establecer una referencia inicial de rendimiento e identificar un modelo candidato sobre el que desarrollar el resto del análisis.

Se evaluaron modelos lineales, representados por la regresión logística con regularización L1 y L2, junto con modelos basados en árboles, como Random Forest y XGBoost. En los modelos de *scikit-learn* que lo permiten, se utilizó el parámetro `class_weight="balanced"` para compensar el desbalanceo de clases [23, 24]. En XGBoost se empleó el parámetro `scale_pos_weight`, calculado en cada fold como la razón entre el número de observaciones negativas y positivas del conjunto de entrenamiento [25].

Tabla 5.4: Resumen de métricas de validación cruzada de los modelos base.

| Modelo        | Accuracy         | Precision        | Recall           | F1               | ROC-AUC          | PR-AUC           |
|---------------|------------------|------------------|------------------|------------------|------------------|------------------|
| Log. Reg. L1  | 68.89 $\pm$ 0.39 | 15.97 $\pm$ 0.37 | 66.96 $\pm$ 2.28 | 25.78 $\pm$ 0.64 | 74.20 $\pm$ 0.88 | 22.05 $\pm$ 0.36 |
| Log. Reg. L2  | 68.78 $\pm$ 0.38 | 15.87 $\pm$ 0.38 | 66.68 $\pm$ 2.28 | 25.63 $\pm$ 0.65 | 74.14 $\pm$ 0.90 | 21.98 $\pm$ 0.33 |
| Random Forest | 77.93 $\pm$ 0.38 | 19.11 $\pm$ 0.59 | 53.64 $\pm$ 1.82 | 28.18 $\pm$ 0.87 | 74.15 $\pm$ 0.49 | 22.30 $\pm$ 0.86 |
| XGBoost       | 71.88 $\pm$ 0.35 | 17.12 $\pm$ 0.29 | 64.63 $\pm$ 1.11 | 27.07 $\pm$ 0.44 | 74.68 $\pm$ 0.43 | 23.31 $\pm$ 0.84 |

*Nota:* las métricas se expresan en porcentaje como media  $\pm$  desviación estándar en validación cruzada.

Los resultados muestran que los modelos lineales alcanzaron valores relativamente elevados de *recall*, aunque mantuvieron una *precision* reducida. Este comportamiento es coherente con un problema de clases desbalanceadas, donde aumentar la capacidad de detección suele implicar un incremento del número de falsos positivos [3, 4].

Por su parte, Random Forest mejoró ligeramente la *precision*, mientras que XGBoost obtuvo los mejores resultados en PR-AUC y ROC-AUC, manteniendo además un equilibrio competitivo entre *precision* y *recall*. Por este motivo, se seleccionó como modelo candidato para evaluar estrategias específicas frente al desbalanceo y desarrollar el análisis posterior del *score*.

Debe tenerse en cuenta que las métricas de *accuracy*, *precision*, *recall* y F1 mostradas en esta comparación inicial se calcularon a partir de la predicción por defecto de cada clasificador, equivalente al uso del umbral estándar de 0.50 en clasificación binaria, al no definirse en esta fase un umbral operativo específico. Por ello, estas métricas se emplean únicamente como referencia diagnóstica.

En cambio, PR-AUC y ROC-AUC evalúan la capacidad de ordenación del modelo sin depender de un umbral fijo. Dado que el objetivo principal del trabajo es construir un *score* de riesgo para ordenar clientes, la interpretación posterior se centra principalmente en la capacidad de ranking del modelo, evaluada mediante PR-AUC, ROC-AUC, deciles, *lift* y captura acumulada de impagos.

## 5.7. Evaluación de estrategias frente al desbalanceo en XGBoost

Una vez seleccionado XGBoost como modelo candidato, se evaluaron distintas estrategias para mejorar el tratamiento de la clase minoritaria.

En primer lugar, se analizaron técnicas de remuestreo, incluyendo Random Oversampling, Random Undersampling y SMOTE. Estas técnicas modifican la distribución de clases durante el entrenamiento, aumentando la representación de la clase minoritaria, reduciendo la clase mayoritaria o generando nuevas observaciones sintéticas [3, 13].

De forma complementaria, también se evaluaron distintas configuraciones del parámetro `scale_pos_weight`. Como punto de partida se utilizó el valor recomendado por la documentación oficial de XGBoost, calculado como la razón entre observaciones negativas y positivas del conjunto de entrenamiento. Posteriormente se analizaron distintas ponderaciones de dicho valor con el objetivo de encontrar un mejor equilibrio entre *precision* y *recall* [25].

La Tabla 5.5 resume únicamente la configuración con mayor PR-AUC medio obtenida dentro de cada familia de estrategias evaluadas. Los resultados completos de todas las configuraciones analizadas se incluyen en el Anexo A.

Tabla 5.5: Comparación resumida de estrategias frente al desbalanceo en XGBoost.

| Estrategia                           | PR-AUC       | Precision     | Recall       | F1           | ROC-AUC      |
|--------------------------------------|--------------|---------------|--------------|--------------|--------------|
| <code>scale_pos_weight</code> = 0.75 | 23.45 ± 0.81 | 19.90 ± 0.16  | 54.51 ± 1.05 | 29.15 ± 0.26 | 74.78 ± 0.48 |
| XGBoost base                         | 23.31 ± 0.84 | 17.12 ± 0.29  | 64.63 ± 1.11 | 27.07 ± 0.44 | 74.68 ± 0.43 |
| Random Oversampling                  | 23.38 ± 0.76 | 31.24 ± 1.41  | 20.66 ± 1.09 | 24.86 ± 1.07 | 74.52 ± 0.42 |
| Random Undersampling                 | 22.88 ± 0.42 | 36.14 ± 2.72  | 11.40 ± 0.71 | 17.32 ± 1.07 | 74.82 ± 0.50 |
| SMOTE                                | 22.97 ± 0.75 | 57.75 ± 14.38 | 0.96 ± 0.30  | 1.89 ± 0.58  | 74.55 ± 0.50 |

*Nota:* se muestra la mejor configuración de cada familia según PR-AUC medio. Las métricas se expresan en porcentaje como media ± desviación estándar.

Los resultados muestran que las técnicas de remuestreo modifican de forma importante el equilibrio entre *precision* y *recall*. Por ejemplo, SMOTE consiguió valores elevados de *precision*, pero redujo considerablemente la capacidad de detectar clientes con impago. En cambio, el modelo base mantuvo un *recall* elevado, aunque con una *precision* inferior.

En este contexto, una *precision* cercana al 20 % no debe interpretarse de forma aislada, ya que la tasa media de impago del conjunto de datos es aproximadamente del 8 %. Por tanto, los segmentos priorizados por el modelo concentran una proporción de impagos claramente superior a la media, lo que justifica analizar su utilidad mediante deciles, *lift* y captura acumulada.

La configuración basada en una ponderación equivalente al 75 % del valor base de `scale_pos_weight` obtuvo el mayor PR-AUC y ROC-AUC entre las estrategias comparadas. Además, mejoró la *precision* respecto al modelo base sin reducir el *recall* por debajo del 50 %, obteniendo también el mejor valor de F1. Por este motivo, se seleccionó como configuración candidata para la fase de ajuste de hiperparámetros.

## 5.8. Selección, ajuste y análisis de umbrales del modelo seleccionado

A partir de la comparación inicial de modelos y del análisis de estrategias frente al desbalanceo, XGBoost con ponderación moderada de la clase positiva se mantuvo como la alternativa más adecuada. La selección de esta configuración no se basó únicamente en una métrica aislada, sino en la combinación del PR-AUC, el ROC-AUC, el F1, el equilibrio entre *precision* y *recall*, la estabilidad en validación cruzada y su utilidad posterior como herramienta de ranking.

En concreto, la configuración seleccionada utilizó un valor de `scale_pos_weight` equivalente al 75 % del peso base, calculado en cada fold como la razón entre observaciones negativas y positivas. Esta alternativa obtuvo el mejor comportamiento global entre las estrategias comparadas, al mejorar la *precision* respecto al modelo base sin reducir el *recall* por debajo del 50 %, además de alcanzar los mayores valores de PR-AUC, ROC-AUC y F1.

Posteriormente se evaluaron distintas configuraciones de ajuste sobre este modelo candidato. Como se observa en la Tabla 5.6, las diferencias entre configuraciones fueron reducidas tanto en métricas globales como en capacidad de concentración de impagos en el primer decil. Aunque los árboles menos profundos alcanzaron una tasa de impago ligeramente superior en el primer decil y una mayor captura en el top 10 %, la configuración base obtuvo el mayor PR-AUC y mantuvo valores muy competitivos en ROC-AUC y en las métricas de ranking. Por ello, se consideró la alternativa más equilibrada para el objetivo del trabajo.

Tabla 5.6: Comparación de desempeño de scoring por configuración XGBoost.

| Configuración                        | PR-AUC | ROC-AUC | Decil 1 | Lift D1 | Captura top 10 % |
|--------------------------------------|--------|---------|---------|---------|------------------|
| Árboles menos profundos              | 23.37  | 74.70   | 26.38   | 3.27    | 32.67            |
| Muestreo más regularizado            | 23.27  | 74.85   | 26.35   | 3.26    | 32.64            |
| Configuración base                   | 23.45  | 74.78   | 26.28   | 3.25    | 32.55            |
| Más árboles y menor tasa aprendizaje | 23.29  | 74.80   | 26.10   | 3.23    | 32.33            |
| Árboles más profundos                | 22.96  | 74.41   | 26.05   | 3.23    | 32.27            |

*Nota:* PR-AUC, ROC-AUC, Decil 1 y Captura top 10 % se expresan en porcentaje. El lift del primer decil indica cuántas veces supera la tasa de impago del decil 1 a la tasa media de impago.

Se priorizó, por tanto, una configuración simple y estable, capaz de mantener una buena capacidad de ordenación del riesgo sin introducir una complejidad adicional que no aportaba una mejora operativa clara. Esta decisión es coherente con el objetivo del trabajo, ya que el interés principal no es únicamente maximizar una métrica aislada, sino obtener un score útil para priorizar clientes según su riesgo relativo de impago [1, 21].

Una vez seleccionada la configuración final del modelo, se analizó el efecto de modificar

el umbral de clasificación sobre las métricas de *precision*, *recall*, F1 y *accuracy*. Este análisis permite mostrar que el comportamiento del modelo no depende únicamente de las probabilidades estimadas, sino también de la decisión operativa sobre qué punto de corte utilizar [6, 4].

El análisis de umbrales se realizó sobre el conjunto de desarrollo utilizando las predicciones obtenidas durante la validación cruzada. Por tanto, el conjunto de test reservado no se utilizó en esta etapa y permaneció completamente aislado para la evaluación final del rendimiento.

En umbrales bajos, el modelo clasifica más clientes como positivos, lo que incrementa el *recall*, pero reduce la *precision* al incorporar más falsos positivos. En cambio, al elevar el umbral, el modelo se vuelve más selectivo: disminuye el número de clientes señalados como riesgosos, pero aumenta la proporción de aciertos dentro de ese grupo.

Tabla 5.7: Métricas del modelo XGBoost final para distintos umbrales de clasificación.

| Umbral | Precision | Recall | F1    | Accuracy |
|--------|-----------|--------|-------|----------|
| 0.30   | 12.75     | 81.76  | 22.05 | 53.34    |
| 0.35   | 14.32     | 75.81  | 24.09 | 61.43    |
| 0.40   | 15.99     | 69.22  | 25.98 | 68.17    |
| 0.45   | 17.84     | 62.00  | 27.71 | 73.88    |
| 0.50   | 19.90     | 54.51  | 29.16 | 78.62    |
| 0.55   | 21.73     | 45.87  | 29.49 | 82.30    |
| 0.60   | 24.80     | 38.77  | 30.25 | 85.57    |
| 0.65   | 27.09     | 30.04  | 28.49 | 87.82    |
| 0.70   | 30.63     | 21.90  | 25.54 | 89.69    |

*Nota:* las métricas se expresan en porcentaje, excepto el umbral de clasificación.

La Tabla 5.7 muestra este compromiso. A medida que el umbral aumenta, la *precision* mejora progresivamente, mientras que el *recall* disminuye. El F1 alcanza sus valores más altos en umbrales intermedios, lo que refleja el equilibrio entre ambas métricas. Por tanto, la elección del punto de corte no debe entenderse como una decisión puramente estadística, sino como una decisión dependiente del objetivo operativo.

Si el objetivo fuera detectar más clientes con posible impago, podría utilizarse un umbral más bajo, aceptando un mayor número de falsos positivos. En cambio, si la capacidad de revisión fuera limitada, podría emplearse un umbral más exigente para priorizar alertas más selectivas.

En la práctica, el modelo final debe entenderse como una herramienta de apoyo a la decisión. Su valor no está en reemplazar completamente una política de crédito, sino en proporcionar una puntuación de riesgo que ayude a priorizar casos. Esta puntuación puede combinarse posteriormente con criterios de negocio, límites operativos y políticas internas de revisión.

## 5.9. Evaluación final del modelo sobre el conjunto de test

Una vez seleccionada la configuración final del modelo, se realizó una evaluación sobre el conjunto de test reservado, compuesto por 267 511 observaciones que no participaron en ninguna etapa del desarrollo del modelo. Este conjunto permaneció aislado durante la comparación de modelos, el tratamiento del desbalanceo, el ajuste de hiperparámetros y la selección del modelo candidato, por lo que proporciona una estimación más realista de la capacidad de generalización del modelo [14, 22].

Tabla 5.8: Evaluación final del modelo seleccionado sobre el conjunto de test.

| Configuración      | PR-AUC | ROC-AUC | Decil 1 | Lift D1 | Captura top 10 % |
|--------------------|--------|---------|---------|---------|------------------|
| Configuración base | 23.32  | 74.90   | 26.40   | 3.27    | 32.70            |

*Nota:* PR-AUC, ROC-AUC, Decil 1 y Captura top 10 % se expresan en porcentaje. El lift del primer decil indica cuántas veces supera la tasa de impago del decil 1 a la tasa media de impago.

La Tabla 5.8 resume el rendimiento obtenido sobre el conjunto de test. El modelo alcanza un PR-AUC de 23,32 % y un ROC-AUC de 74,90 %, valores coherentes con los obtenidos durante la validación cruzada sobre el conjunto de desarrollo. Esto sugiere que el modelo mantiene una capacidad estable para discriminar entre clientes con y sin impago al aplicarse sobre datos no utilizados previamente.

Además de las métricas globales, se evaluó la capacidad del modelo para concentrar los casos de impago en los segmentos de mayor riesgo. En el primer decil, formado por el 10 % de clientes con mayor puntuación estimada, la tasa observada de impago fue del 26,40 %, frente a una tasa media cercana al 8,07 % en el conjunto de test. Esto equivale a un *lift* de 3,27, es decir, el grupo de mayor riesgo presenta una concentración de impagos más de tres veces superior al promedio.

Tabla 5.9: Captura acumulada de impagos en los principales grupos de riesgo sobre el conjunto de test.

| Grupo de riesgo | Clientes acum. | Impagos acum. | Captura (%) | Lift acum. |
|-----------------|----------------|---------------|-------------|------------|
| Top 10 %        | 26 751         | 7 062         | 32.70       | 3.27       |
| Top 20 %        | 53 502         | 10 981        | 50.85       | 2.54       |
| Top 30 %        | 80 253         | 13 718        | 63.52       | 2.12       |

*Nota:* la captura indica el porcentaje acumulado de impagos identificados dentro de cada grupo de mayor riesgo estimado por el modelo. El lift acumulado compara dicha concentración con la tasa media de impago del conjunto de test.

La Tabla 5.9 muestra la captura acumulada de impagos en los principales grupos de riesgo. Al revisar únicamente el 10 % de clientes con mayor puntuación, el modelo identifica el 32,70 % de todos los impagos del conjunto de test. Si la capacidad de revisión aumenta al 20 % y al 30 % de clientes, la captura acumulada se eleva al 50,85 % y al 63,52 %, respectivamente.

Estos resultados muestran que la utilidad del modelo no se limita a una clasificación binaria directa. Su principal aportación consiste en ordenar a los clientes según su riesgo relativo, permitiendo priorizar la revisión de aquellos casos donde la probabilidad estimada de impago es mayor.

## 5.10. Evaluación del scoring: deciles, lift y captura acumulada

Para analizar con mayor detalle el comportamiento del modelo como herramienta de scoring, los clientes del conjunto de test fueron ordenados de mayor a menor probabilidad estimada de impago y divididos en deciles de riesgo. Este análisis permite comprobar si el modelo concentra una mayor proporción de clientes con impago en los segmentos superiores del ranking [1, 21].

Tabla 5.10: Análisis de deciles del modelo XGBoost final sobre el conjunto de test.

| Decil riesgo | N.º clientes | N.º impagos | Tasa de impago (%) | Lift | Captura acum. (%) |
|--------------|--------------|-------------|--------------------|------|-------------------|
| 1            | 26 751       | 7 062       | 26.40              | 3.27 | 32.70             |
| 2            | 26 751       | 3 919       | 14.65              | 1.81 | 50.85             |
| 3            | 26 751       | 2 737       | 10.23              | 1.27 | 63.52             |
| 4            | 26 751       | 2 126       | 7.95               | 0.98 | 73.37             |
| 5            | 26 751       | 1 651       | 6.17               | 0.76 | 81.01             |
| 6            | 26 751       | 1 321       | 4.94               | 0.61 | 87.13             |
| 7            | 26 751       | 1 068       | 3.99               | 0.49 | 92.07             |
| 8            | 26 751       | 787         | 2.94               | 0.36 | 95.72             |
| 9            | 26 751       | 569         | 2.13               | 0.26 | 98.35             |
| 10           | 26 752       | 356         | 1.33               | 0.16 | 100.00            |

*Nota:* los deciles están ordenados de mayor a menor riesgo estimado por el modelo. La captura acumulada indica el porcentaje de impagos concentrado hasta cada decil.

Los resultados de la Tabla 5.10 muestran una relación decreciente entre el decil de riesgo y la tasa observada de impago. El primer decil presenta una tasa de impago del 26,40 %, mientras que el segundo y el tercer decil alcanzan tasas del 14,65 % y 10,23 %, respectivamente. Estos tres primeros grupos se sitúan por encima de la tasa media del conjunto de test, lo que confirma que el modelo concentra los casos de mayor riesgo en las primeras posiciones del ranking.

A partir del cuarto decil, la tasa de impago se aproxima o queda por debajo de la tasa media, y continúa disminuyendo progresivamente hasta el último decil. Este patrón es consistente con el objetivo del scoring: asignar mayores puntuaciones a los clientes con mayor probabilidad relativa de impago y menores puntuaciones a los clientes de menor riesgo.

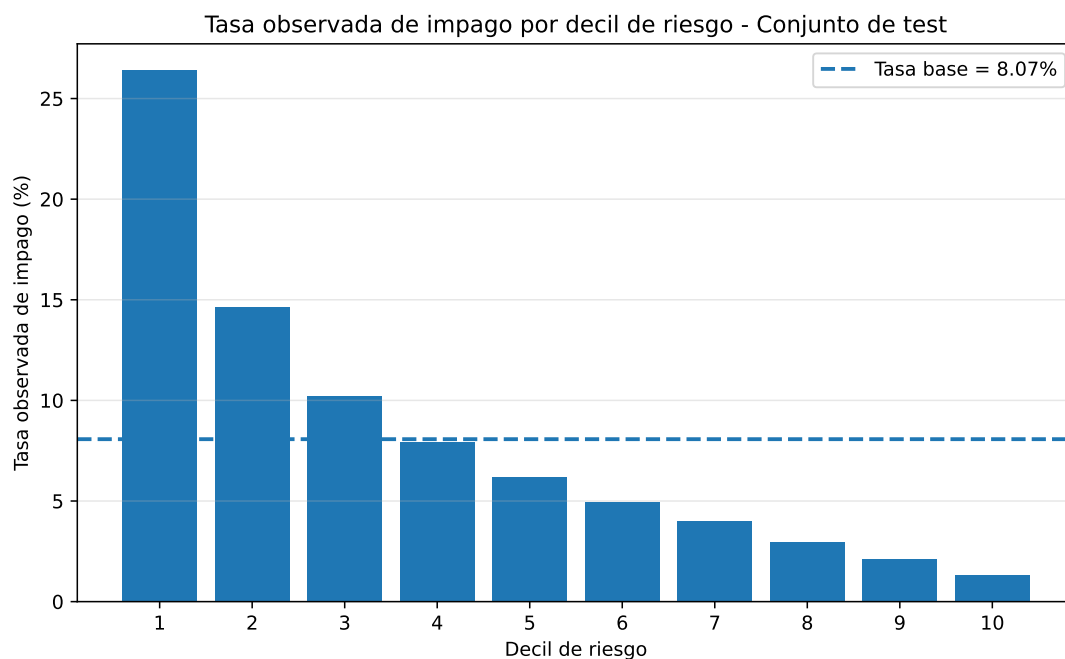


Figura 5.1: Tasa observada de impago por decil de riesgo en el conjunto de test.

La Figura 5.1 refuerza visualmente este comportamiento. Los tres primeros deciles presentan tasas de impago superiores a la tasa base, mientras que los deciles inferiores concentran una proporción mucho menor de casos positivos. Este patrón muestra que el modelo no asigna el riesgo de forma aleatoria, sino que tiende a concentrar los casos de impago en los segmentos con mayor probabilidad estimada. Por tanto, el análisis por deciles y *lift* confirma que el modelo aporta valor como herramienta de priorización en procesos de evaluación crediticia.

Los resultados obtenidos en el conjunto de test indican que el modelo XGBoost seleccionado mantiene un comportamiento estable fuera del conjunto de desarrollo. No se observa una caída relevante en las métricas globales ni en la capacidad de concentración de impagos en los segmentos de mayor riesgo, lo que sugiere que el proceso de selección y ajuste no produjo un sobreajuste significativo.

En un problema de impago con clases fuertemente desbalanceadas, la interpretación de los resultados debe ir más allá de la *accuracy*. Una alta exactitud global puede ser poco informativa si el modelo acierta principalmente sobre la clase mayoritaria. Por ello, métricas como PR-AUC, *precision*, *recall*, deciles, lift y captura acumulada ofrecen una visión más adecuada del rendimiento sobre la clase de interés [3, 6, 4].

Desde una perspectiva aplicada, el modelo debe entenderse como una herramienta de apoyo a la decisión y no como un mecanismo automático de concesión o rechazo de créditos. Su utilidad principal consiste en proporcionar una puntuación de riesgo que permita ordenar clientes, priorizar revisiones y orientar recursos hacia los casos con mayor probabilidad estimada de impago.

No obstante, el modelo también presenta limitaciones. La baja frecuencia de la clase positiva dificulta alcanzar simultáneamente valores elevados de *precision* y *recall*. Un

umbral más exigente podría aumentar la proporción de verdaderos impagos entre los clientes señalados como riesgosos, pero también reduciría la cantidad total de impagos detectados. Por tanto, el umbral operativo final debería definirse considerando los costes asociados a falsos positivos y falsos negativos, así como la capacidad real de revisión disponible en un contexto de negocio [1, 21].



## 6. Conclusiones

### 6.1. Principales resultados

El presente trabajo tuvo como objetivo desarrollar y evaluar modelos de *machine learning* para la predicción del riesgo de impago de crédito utilizando el conjunto de datos *Home Credit Default Risk*. Aunque el problema se formuló como una tarea de clasificación binaria, el enfoque aplicado se centró en analizar la utilidad de las probabilidades estimadas como una herramienta de *scoring*, capaz de ordenar a los clientes según su riesgo relativo de impago.

Una característica central del problema fue el fuerte desbalanceo de clases, ya que la clase positiva representa una proporción reducida del total de clientes. Por este motivo, la evaluación no se basó únicamente en la *accuracy*, sino también en métricas más adecuadas para este contexto, como PR-AUC, ROC-AUC, *precision*, *recall* y F1. Asimismo, se incorporaron métricas de ranking, como deciles de riesgo, *lift* y captura acumulada de impagos, con el fin de evaluar la capacidad del modelo para priorizar correctamente los casos de mayor riesgo.

Desde el punto de vista metodológico, se construyó un conjunto de datos integrado a partir de la tabla principal `application_train` y de información histórica procedente de `bureau` y `previous_application`. Estas tablas auxiliares fueron agregadas a nivel de cliente, manteniendo una estructura de una fila por solicitud. Además, se reservaron 267 511 observaciones como conjunto de test final, que no participaron en la comparación de modelos, el tratamiento del desbalanceo ni el ajuste de hiperparámetros.

La comparación inicial permitió seleccionar XGBoost como modelo candidato principal, al presentar el mejor rendimiento en PR-AUC y ROC-AUC dentro de los modelos base evaluados. Posteriormente, se analizaron distintas estrategias frente al desbalanceo y varias configuraciones del modelo. La configuración final mantuvo una ponderación moderada de la clase positiva, equivalente al 75 % del valor base de `scale_pos_weight`, y se conservó la configuración base de XGBoost por ofrecer un buen equilibrio entre rendimiento, estabilidad y simplicidad.

En el conjunto de test, el modelo obtuvo un PR-AUC de 23,32 % y un ROC-AUC de 74,90 %. Además, el primer decil de riesgo alcanzó una tasa de impago del 26,40 %, frente a una tasa media cercana al 8,07 %, lo que equivale a un *lift* de 3,27. En términos de captura acumulada, el modelo identificó el 32,70 % de todos los impagos revisando únicamente el 10 % de clientes con mayor puntuación, y el 50,85 % al ampliar la revisión al 20 %. Estos resultados confirman que el modelo concentra una proporción relevante de impagos en los segmentos de mayor riesgo.

## 6.2. Utilidad práctica del modelo

Desde una perspectiva aplicada, el modelo desarrollado debe entenderse como una herramienta de apoyo a la decisión. Su principal utilidad no consiste en aprobar o rechazar créditos de forma automática, sino en proporcionar una puntuación de riesgo que permita priorizar revisiones y orientar los recursos de análisis hacia los clientes con mayor probabilidad estimada de impago.

Esta interpretación resulta especialmente importante debido al desbalanceo de clases y a la *precision* limitada obtenida. En un contexto de impago, utilizar el modelo como una regla automática podría generar un número elevado de falsos positivos. En cambio, su uso como herramienta de *scoring* permite establecer distintos niveles de revisión según la política de riesgo, la capacidad operativa disponible y el coste relativo de cada tipo de error.

El análisis por deciles y *lift* muestra que el modelo distingue segmentos de riesgo de forma útil: los primeros deciles presentan tasas de impago superiores a la media, mientras que los deciles inferiores concentran una proporción menor de casos positivos. Por tanto, el modelo puede aportar valor como una primera capa de priorización dentro de un proceso de evaluación crediticia, siempre complementado por criterios de negocio y revisión humana.

## 6.3. Limitaciones del estudio

A pesar de los resultados obtenidos, el trabajo presenta algunas limitaciones. En primer lugar, el conjunto de datos utilizado corresponde a una base histórica concreta, por lo que los resultados no pueden generalizarse automáticamente a cualquier entidad financiera o cartera de crédito. Antes de aplicar un modelo de este tipo en un entorno real, sería necesario validarlo con datos propios, actuales y representativos de la población objetivo.

En segundo lugar, aunque el modelo permite ordenar clientes de forma útil, las probabilidades estimadas no fueron calibradas de forma específica. Esto significa que el modelo puede ser adecuado para construir un ranking de riesgo, pero no necesariamente garantiza que cada probabilidad estimada coincida exactamente con la tasa real de impago esperada. Esta calibración sería especialmente relevante si el *score* se quisiera utilizar directamente en políticas cuantitativas de decisión [26].

Otra limitación relevante es que la selección del umbral no incorporó una matriz explícita de costes. En la práctica, los falsos positivos y falsos negativos tienen impactos económicos distintos, por lo que el punto de corte final debería definirse considerando tanto métricas estadísticas como criterios de negocio.

Asimismo, el trabajo no abordó en profundidad la explicabilidad individual de las predicciones ni la evaluación temporal del modelo. En un entorno regulado, especialmente en crédito, puede ser necesario justificar por qué un cliente recibe una determinada puntuación de riesgo. Además, el comportamiento del riesgo de crédito puede cambiar con el tiempo debido a factores económicos, laborales o regulatorios, por lo que sería necesario monitorizar el rendimiento del modelo de forma periódica [27].

## 6.4. Líneas futuras de investigación

A partir de las limitaciones anteriores, una primera línea futura sería aplicar técnicas de calibración de probabilidades, como calibración isotónica o sigmoïdal. Esto permitiría comprobar si las probabilidades estimadas por el modelo se corresponden adecuadamente con las tasas reales de impago observadas [26].

Una segunda línea de mejora consistiría en incorporar una matriz de costes asociada a falsos positivos y falsos negativos. De esta forma, la selección del umbral podría realizarse no solo en función de métricas como F1, *precision* o *recall*, sino también desde una perspectiva económica y operativa.

También sería recomendable incorporar técnicas de interpretabilidad avanzada, como SHAP, para analizar la contribución de cada variable a las predicciones del modelo. Esto permitiría complementar el análisis global del rendimiento con explicaciones individuales, facilitando la revisión de casos concretos y aumentando la transparencia del sistema de *scoring* [28].

Otra línea futura relevante sería evaluar posibles sesgos o impactos desiguales entre grupos de clientes. En crédito, las decisiones deben ser justificables, trazables y compatibles con criterios regulatorios y de equidad, por lo que sería necesario analizar si las predicciones afectan de forma desproporcionada a determinados perfiles [29].

Finalmente, sería conveniente estudiar la estabilidad temporal del modelo. Una futura investigación podría evaluar el rendimiento del *score* en diferentes periodos o bajo distintas condiciones económicas, con el fin de definir estrategias de monitorización, recalibración y mantenimiento del modelo a lo largo del tiempo.

En conjunto, estas líneas futuras permitirían avanzar desde un modelo de *scoring* evaluado en un entorno académico hacia una herramienta más robusta, interpretable y cercana a una aplicación real en la gestión del riesgo crediticio.

# A. Resultados completos de estrategias frente al desbalanceo

En este anexo se presentan los resultados completos de las configuraciones evaluadas frente al desbalanceo de clases para el modelo XGBoost. En el cuerpo principal del trabajo se muestra únicamente una versión resumida con la mejor configuración de cada familia de estrategias.

Tabla A.1: Comparación de estrategias frente al desbalanceo para XGBoost.

| Modelo              | Accuracy | Precision | Recall | F1    | ROC-AUC | PR-AUC |
|---------------------|----------|-----------|--------|-------|---------|--------|
| XGB weight 0.75     | 78.62    | 19.90     | 54.51  | 29.15 | 74.78   | 23.45  |
| XGB + ROS 0.30      | 89.92    | 31.24     | 20.66  | 24.86 | 74.52   | 23.38  |
| XGBoost             | 71.88    | 17.12     | 64.63  | 27.07 | 74.68   | 23.31  |
| XGB + ROS 0.20      | 91.35    | 37.27     | 10.44  | 16.29 | 74.67   | 23.27  |
| XGB + ROS 0.50      | 85.39    | 24.24     | 38.06  | 29.61 | 74.60   | 23.17  |
| XGB + SMOTE 0.20 k5 | 91.94    | 57.75     | 0.96   | 1.89  | 74.55   | 22.97  |
| XGB + SMOTE 0.20 k3 | 91.94    | 54.81     | 0.81   | 1.59  | 74.52   | 22.96  |
| XGB + RUS 0.20      | 91.22    | 36.14     | 11.40  | 17.32 | 74.82   | 22.88  |
| XGB + RUS 0.30      | 89.46    | 29.82     | 22.61  | 25.71 | 74.61   | 22.80  |
| XGB + RUS 0.50      | 84.07    | 22.85     | 41.00  | 29.34 | 74.44   | 22.64  |
| XGB + SMOTE 0.30 k5 | 91.94    | 59.36     | 0.77   | 1.53  | 74.40   | 22.54  |
| XGB + SMOTE 0.30 k3 | 91.93    | 57.34     | 0.81   | 1.59  | 74.36   | 22.44  |
| XGB + SMOTE 0.50 k3 | 91.93    | 48.57     | 0.65   | 1.28  | 74.06   | 21.89  |
| XGB + SMOTE 0.50 k5 | 91.94    | 54.48     | 0.62   | 1.22  | 74.17   | 21.73  |

*Nota:* las métricas se expresan en porcentaje. ROS hace referencia a *Random Over-Sampling*, RUS a *Random Under-Sampling* y SMOTE a *Synthetic Minority Over-sampling Technique*.

## B. Código relevante del capítulo 5

Este anexo recoge únicamente los fragmentos de código más representativos del proceso experimental descrito en el Capítulo 5. No se incluye el cuaderno completo, sino las partes necesarias para reproducir la preparación del conjunto de desarrollo, el preprocesamiento, la validación cruzada, el ajuste de XGBoost y el análisis posterior de umbrales, deciles, *lift* y captura acumulada.

### B.1. Partición estratificada y creación de variables

```
1  # Train-test split
2  df_train, df_test = train_test_split(
3      df1,
4      train_size=40000,
5      stratify=df1["TARGET"],
6      random_state=42
7  )
8
9  df_train = df_train.copy()
10 df_test = df_test.copy()
11
12 def create_features(data):
13     data = data.copy()
14
15     # History indicators
16     data["has_bureau_history"] = data["n_bureau_credits"].notna().astype(int)
17     data["has_previous_application_history"] = (
18         data["n_previous_applications"].notna().astype(int)
19     )
20
21     # Count variables from external tables
22     history_count_cols = [
23         "n_bureau_credits", "n_active_credits", "n_closed_credits",
24         "n_previous_applications", "n_approved_previous",
25         "n_refused_previous"
26     ]
27     data[history_count_cols] = data[history_count_cols].fillna(0)
28
29     # Time variables
30     data["AGE_YEARS"] = data["DAYS_BIRTH"] / -365
31     data["DAYS_EMPLOYED_ANOMALY"] = (
32         data["DAYS_EMPLOYED"] == 365243
33     ).astype(int)
34     data["DAYS_EMPLOYED_CLEAN"] = data["DAYS_EMPLOYED"].replace(365243, np.nan)
35     data["YEARS_EMPLOYED"] = data["DAYS_EMPLOYED_CLEAN"] / -365
36     data["YEARS_REGISTRATION"] = data["DAYS_REGISTRATION"] / -365
37     data["YEARS_ID_PUBLISH"] = data["DAYS_ID_PUBLISH"] / -365
38     data["YEARS_LAST_PHONE_CHANGE"] = data["DAYS_LAST_PHONE_CHANGE"] / -365
39
40     # External source summary
41     ext_cols = ["EXT_SOURCE_1", "EXT_SOURCE_2", "EXT_SOURCE_3"]
42     data["EXT_SOURCE_MEAN"] = data[ext_cols].mean(axis=1)
43     data["EXT_SOURCE_MIN"] = data[ext_cols].min(axis=1)
44     data["EXT_SOURCE_MAX"] = data[ext_cols].max(axis=1)
45     data["EXT_SOURCE_COUNT"] = data[ext_cols].notna().sum(axis=1)
46
```

```

47     # Financial ratios
48     data["CREDIT_INCOME_RATIO"] = (
49         data["AMT_CREDIT"] / data["AMT_INCOME_TOTAL"]
50     )
51     data["ANNUITY_INCOME_RATIO"] = (
52         data["AMT_ANNUITY"] / data["AMT_INCOME_TOTAL"]
53     )
54
55     # Bureau request variables
56     recent_cols = [
57         "AMT_REQ_CREDIT_BUREAU_HOUR",
58         "AMT_REQ_CREDIT_BUREAU_DAY",
59         "AMT_REQ_CREDIT_BUREAU_WEEK"
60     ]
61     total_cols = recent_cols + [
62         "AMT_REQ_CREDIT_BUREAU_MON",
63         "AMT_REQ_CREDIT_BUREAU_QRT",
64         "AMT_REQ_CREDIT_BUREAU_YEAR"
65     ]
66
67     data["BUREAU_REQ_RECENT"] = data[recent_cols].sum(axis=1, min_count=1)
68     data["BUREAU_REQ_TOTAL"] = data[total_cols].sum(axis=1, min_count=1)
69
70     return data
71
72 df_train = create_features(df_train)
73 df_test = create_features(df_test)
74
75 df_train = df_train.drop(columns=["SK_ID_CURR"])
76 df_test = df_test.drop(columns=["SK_ID_CURR"])

```

## B.2. Preprocesamiento dentro de cada fold

```

1  def preprocess_fold(X_train, X_valid):
2      X_train = X_train.copy()
3      X_valid = X_valid.copy()
4
5      num_cols = X_train.select_dtypes(
6          include=["int64", "float64"]
7      ).columns.tolist()
8
9      cat_cols = X_train.select_dtypes(
10         include=["object", "category", "bool"]
11     ).columns.tolist()
12
13     # Missing indicators and median imputation
14     for col in num_cols:
15         if X_train[col].isna().sum() > 0:
16             X_train[col + "_missing"] = X_train[col].isna().astype(int)
17             X_valid[col + "_missing"] = X_valid[col].isna().astype(int)
18
19             median_value = X_train[col].median()
20             X_train[col] = X_train[col].fillna(median_value)
21             X_valid[col] = X_valid[col].fillna(median_value)
22
23     # Categorical imputation and rare categories
24     for col in cat_cols:
25         X_train[col] = X_train[col].fillna("Missing")
26         X_valid[col] = X_valid[col].fillna("Missing")
27
28         if not col.endswith("_GROUP"):
29             freq = X_train[col].value_counts(normalize=True)

```

```

30         common_categories = freq[freq >= 0.01].index
31
32         X_train[col] = X_train[col].where(
33             X_train[col].isin(common_categories), "Other"
34         )
35         X_valid[col] = X_valid[col].where(
36             X_valid[col].isin(common_categories), "Other"
37         )
38
39         # One-hot encoding and column alignment
40         X_train = pd.get_dummies(X_train, drop_first=False)
41         X_valid = pd.get_dummies(X_valid, drop_first=False)
42
43         X_train, X_valid = X_train.align(
44             X_valid,
45             join="left",
46             axis=1,
47             fill_value=0
48         )
49
50         return X_train, X_valid

```

### B.3. Validación cruzada y métricas de evaluación

```

1  skf = StratifiedKFold(
2      n_splits=5,
3      shuffle=True,
4      random_state=42
5  )
6
7  def evaluate_model_cv(model, X, y, model_name, scale_data=False):
8      results = []
9      confusion_matrices = []
10
11     for fold, (train_idx, valid_idx) in enumerate(skf.split(X, y), start=1):
12
13         X_train = X.iloc[train_idx].copy()
14         X_valid = X.iloc[valid_idx].copy()
15         y_train = y.iloc[train_idx].copy()
16         y_valid = y.iloc[valid_idx].copy()
17
18         X_train, X_valid = preprocess_fold(X_train, X_valid)
19
20         if scale_data:
21             scaler = StandardScaler()
22             X_train = scaler.fit_transform(X_train)
23             X_valid = scaler.transform(X_valid)
24
25         model.fit(X_train, y_train)
26
27         y_pred = model.predict(X_valid)
28         y_proba = model.predict_proba(X_valid)[: , 1]
29
30         confusion_matrices.append(confusion_matrix(y_valid, y_pred))
31
32         results.append({
33             "model": model_name,
34             "fold": fold,
35             "accuracy": accuracy_score(y_valid, y_pred),
36             "precision": precision_score(y_valid, y_pred, zero_division=0),
37             "recall": recall_score(y_valid, y_pred, zero_division=0),
38             "f1": f1_score(y_valid, y_pred, zero_division=0),

```

```

39         "roc_auc": roc_auc_score(y_valid, y_proba),
40         "pr_auc": average_precision_score(y_valid, y_proba)
41     })
42
43     results_df = pd.DataFrame(results)
44     summary = results_df.drop(columns=["model", "fold"]).agg(["mean", "std"])
45     total_cm = sum(confusion_matrices)
46
47     return results_df, summary, total_cm

```

## B.4. Ajuste práctico del modelo XGBoost

```

1  def evaluate_xgb_cv(model_name, xgb_params, weight_multiplier=1.0):
2      results = []
3      confusion_matrices = []
4      y_valid_all, y_proba_all, valid_index_all = [], [], []
5
6      for fold, (train_idx, valid_idx) in enumerate(skf.split(X, y), start=1):
7
8          X_train = X.iloc[train_idx].copy()
9          X_valid = X.iloc[valid_idx].copy()
10         y_train = y.iloc[train_idx].copy()
11         y_valid = y.iloc[valid_idx].copy()
12
13         X_train, X_valid = preprocess_fold(X_train, X_valid)
14
15         base_weight = (y_train == 0).sum() / (y_train == 1).sum()
16         scale_pos_weight = base_weight * weight_multiplier
17
18         params = xgb_params.copy()
19         params["scale_pos_weight"] = scale_pos_weight
20
21         xgb_model = XGBClassifier(**params)
22         xgb_model.fit(X_train, y_train)
23
24         y_pred = xgb_model.predict(X_valid)
25         y_proba = xgb_model.predict_proba(X_valid)[: , 1]
26
27         confusion_matrices.append(confusion_matrix(y_valid, y_pred))
28
29         results.append({
30             "model": model_name,
31             "fold": fold,
32             "accuracy": accuracy_score(y_valid, y_pred),
33             "precision": precision_score(y_valid, y_pred, zero_division=0),
34             "recall": recall_score(y_valid, y_pred, zero_division=0),
35             "f1": f1_score(y_valid, y_pred, zero_division=0),
36             "roc_auc": roc_auc_score(y_valid, y_proba),
37             "pr_auc": average_precision_score(y_valid, y_proba)
38         })
39
40         y_valid_all.extend(y_valid)
41         y_proba_all.extend(y_proba)
42         valid_index_all.extend(valid_idx)
43
44     results_df = pd.DataFrame(results)
45     summary = results_df.drop(columns=["model", "fold"]).agg(["mean", "std"])
46     total_cm = sum(confusion_matrices)
47
48     oof_predictions = pd.DataFrame({
49         "original_index": valid_index_all,
50         "y_true": y_valid_all,

```



```

51         "y_proba": y_proba_all
52     }).sort_values("original_index").reset_index(drop=True)
53
54     return results_df, summary, total_cm, oof_predictions
55
56
57 xgb_params_base = {
58     "n_estimators": 300,
59     "max_depth": 3,
60     "learning_rate": 0.05,
61     "subsample": 0.8,
62     "colsample_bytree": 0.8,
63     "eval_metric": "aucpr",
64     "random_state": 42,
65     "n_jobs": -1
66 }
67
68 tuning_configs = [
69     {"model": "Base configuration", "params": xgb_params_base},
70     {"model": "More trees lower learning rate",
71      "params": {**xgb_params_base, "n_estimators": 500,
72                "learning_rate": 0.03}},
73     {"model": "Shallower trees",
74      "params": {**xgb_params_base, "max_depth": 2}},
75     {"model": "Deeper trees",
76      "params": {**xgb_params_base, "max_depth": 4}},
77     {"model": "More regularized sampling",
78      "params": {**xgb_params_base, "subsample": 0.7,
79                "colsample_bytree": 0.7}}
80 ]
81
82 tuning_results = []
83 tuning_oof_predictions = {}
84 tuning_confusion_matrices = {}
85
86 for config in tuning_configs:
87
88     results_df, summary, cm, oof_df = evaluate_xgb_cv(
89         model_name=config["model"],
90         xgb_params=config["params"],
91         weight_multiplier=0.75
92     )
93
94     tuning_results.append({
95         "model": config["model"],
96         "accuracy_mean": summary.loc["mean", "accuracy"],
97         "precision_mean": summary.loc["mean", "precision"],
98         "recall_mean": summary.loc["mean", "recall"],
99         "f1_mean": summary.loc["mean", "f1"],
100         "roc_auc_mean": summary.loc["mean", "roc_auc"],
101         "pr_auc_mean": summary.loc["mean", "pr_auc"]
102     })
103
104     tuning_oof_predictions[config["model"]] = oof_df
105     tuning_confusion_matrices[config["model"]] = cm
106
107 tuning_results_df = (
108     pd.DataFrame(tuning_results)
109     .sort_values(
110         by=["pr_auc_mean", "roc_auc_mean", "f1_mean"],
111         ascending=False
112     )
113     .reset_index(drop=True)
114 )

```

## B.5. Umbrales, deciles, lift y captura acumulada

```

1 best_model_name = tuning_results_df.loc[0, "model"]
2 best_oof_df = tuning_oof_predictions[best_model_name].copy()
3
4 # Threshold analysis
5 threshold_results = []
6 thresholds = np.arange(0.30, 0.701, 0.01)
7
8 for threshold in thresholds:
9     y_pred_threshold = (best_oof_df["y_proba"] >= threshold).astype(int)
10
11     threshold_results.append({
12         "threshold": round(threshold, 2),
13         "accuracy": accuracy_score(best_oof_df["y_true"], y_pred_threshold),
14         "precision": precision_score(
15             best_oof_df["y_true"], y_pred_threshold, zero_division=0
16         ),
17         "recall": recall_score(
18             best_oof_df["y_true"], y_pred_threshold, zero_division=0
19         ),
20         "f1": f1_score(
21             best_oof_df["y_true"], y_pred_threshold, zero_division=0
22         )
23     })
24
25 threshold_results_df = pd.DataFrame(threshold_results)
26
27
28 def decile_lift_capture_table(oof_df, model_name):
29     scoring_df = oof_df.copy()
30
31     scoring_df["risk_decile"] = pd.qcut(
32         scoring_df["y_proba"],
33         q=10,
34         labels=False,
35         duplicates="drop"
36     )
37
38     # Decile 1 represents the highest predicted risk
39     scoring_df["risk_decile"] = 10 - scoring_df["risk_decile"]
40
41     base_default_rate = scoring_df["y_true"].mean()
42     total_defaults = scoring_df["y_true"].sum()
43
44     decile_table = (
45         scoring_df
46         .groupby("risk_decile")
47         .agg(
48             n_clients=("y_true", "count"),
49             n_defaults=("y_true", "sum"),
50             default_rate=("y_true", "mean"),
51             min_score=("y_proba", "min"),
52             max_score=("y_proba", "max"),
53             mean_score=("y_proba", "mean")
54         )
55         .reset_index()
56         .sort_values("risk_decile")
57     )
58

```

```
59     decile_table["model"] = model_name
60     decile_table["default_rate_pct"] = decile_table["default_rate"] * 100
61     decile_table["lift"] = decile_table["default_rate"] / base_default_rate
62
63     decile_table["cum_clients"] = decile_table["n_clients"].cumsum()
64     decile_table["cum_defaults"] = decile_table["n_defaults"].cumsum()
65
66     decile_table["cum_client_pct"] = (
67         decile_table["cum_clients"] / decile_table["n_clients"].sum()
68     )
69
70     decile_table["capture_rate"] = (
71         decile_table["cum_defaults"] / total_defaults
72     )
73
74     decile_table["cum_default_rate"] = (
75         decile_table["cum_defaults"] / decile_table["cum_clients"]
76     )
77
78     decile_table["cum_lift"] = (
79         decile_table["cum_default_rate"] / base_default_rate
80     )
81
82     return decile_table
83
84
85 best_decile_table = decile_lift_capture_table(
86     best_oof_df,
87     best_model_name
88 )
```

# Bibliografía

- [1] David J. Hand and William E. Henley. Statistical classification methods in consumer credit scoring: A review. *Journal of the Royal Statistical Society: Series A*, 160(3): 523–541, 1997. doi: 10.1111/j.1467-985X.1997.00078.x.
- [2] Stefan Lessmann, Bart Baesens, Hsin-Vonn Seow, and Lyn C. Thomas. Benchmarking state-of-the-art classification algorithms for credit scoring: An update of research. *European Journal of Operational Research*, 247(1):124–136, 2015. doi: 10.1016/j.ejor.2015.05.030.
- [3] Haibo He and Eduardo A. Garcia. Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9):1263–1284, 2009. doi: 10.1109/TKDE.2008.239.
- [4] Takaya Saito and Marc Rehmsmeier. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3):e0118432, 2015. doi: 10.1371/journal.pone.0118432.
- [5] Tom Fawcett. An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8): 861–874, 2006. doi: 10.1016/j.patrec.2005.10.010.
- [6] Jesse Davis and Mark Goadrich. The relationship between precision-recall and ROC curves. In *Proceedings of the 23rd International Conference on Machine Learning*, pages 233–240, 2006. doi: 10.1145/1143844.1143874.
- [7] Home Credit Group. Home Credit Default Risk. <https://www.kaggle.com/competitions/home-credit-default-risk>, 2018. Kaggle competition. Accedido: 2026-07-07.
- [8] Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. *An Introduction to Statistical Learning: with Applications in R*. Springer, 2 edition, 2021. doi: 10.1007/978-1-0716-1418-1.
- [9] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, 2 edition, 2009. doi: 10.1007/978-0-387-84858-7.
- [10] Peter McCullagh and John A. Nelder. *Generalized Linear Models*. Chapman and Hall, 2 edition, 1989.
- [11] Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970. doi: 10.1080/00401706.1970.10488634.

- [12] Robert Tibshirani. Regression shrinkage and selection via the Lasso. *Journal of the Royal Statistical Society: Series B*, 58(1):267–288, 1996. doi: 10.1111/j.2517-6161.1996.tb02080.x.
- [13] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002. doi: 10.1613/jair.953.
- [14] Ron Kohavi. A study of cross-validation and bootstrap for accuracy estimation and model selection. In *Proceedings of the 14th International Joint Conference on Artificial Intelligence*, pages 1137–1143, 1995.
- [15] Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Wadsworth International Group, 1984.
- [16] Leo Breiman. Bagging predictors. *Machine Learning*, 24(2):123–140, 1996. doi: 10.1007/BF00058655.
- [17] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi: 10.1023/A:1010933404324.
- [18] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001. doi: 10.1214/aos/1013203451.
- [19] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. doi: 10.1145/2939672.2939785.
- [20] Jerome H. Friedman. Stochastic gradient boosting. *Computational Statistics & Data Analysis*, 38(4):367–378, 2002. doi: 10.1016/S0167-9473(01)00065-2.
- [21] Raymond Anderson. *The Credit Scoring Toolkit: Theory and Practice for Retail Credit Risk Management and Decision Automation*. Oxford University Press, 2007.
- [22] Shachar Kaufman, Saharon Rosset, Claudia Perlich, and Ori Stitelman. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4):1–21, 2012. doi: 10.1145/2382577.2382579.
- [23] scikit-learn developers. LogisticRegression. [https://scikit-learn.org/stable/modules/generated/sklearn.linear\\_model.LogisticRegression.html](https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html), 2026. Accedido: 2026-07-07.
- [24] scikit-learn developers. RandomForestClassifier. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>, 2026. Accedido: 2026-07-07.
- [25] XGBoost developers. XGBoost Parameters. <https://xgboost.readthedocs.io/en/stable/parameter.html>, 2026. Accedido: 2026-07-07.

- 
- [26] Alexandru Niculescu-Mizil and Rich Caruana. Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning*, pages 625–632, 2005. doi: 10.1145/1102351.1102430.
  - [27] João Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4):1–37, 2014. doi: 10.1145/2523813. Article 44.
  - [28] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
  - [29] Moritz Hardt, Eric Price, and Nathan Srebro. Equality of opportunity in supervised learning. In *Advances in Neural Information Processing Systems*, volume 29, 2016.